

量子编码大模型研发研究报告

未来院

2026-04-10

目录

重点架构与执行概览 (Executive Summary)	6
1 实验体系总论 (System Overview)	6
1.1 研究目标	6
1.2 模型训练与进展概览	6
2 高维数据链路与评价闭环 (Data & Rigorous Evaluation Bounds)	9
2.1 数据设计原则	9
2.2 主要数据集概览	9
2.3 混合域严格 holdout	9
2.4 量子严格 holdout	11
2.5 prompt family 划分	11
2.6 样本配额	11
2.7 Category 与 Prompt Family 细分	12
2.8 快迭代子集与难例子集	14
2.9 Gemma router warmup 数据	14
2.10 Gemma 路由 Warmup 数据边界	15
2.11 实验纪律	15
2.12 小结	15
3 模型后训练架构：预调与反馈纠正 (Post-training Methods)	16
3.1 算法栈总览	16
3.2 SFT 主干	16
3.3 选择性训练控制	17
3.4 Gemma MoE 微调策略	17
3.5 SFT 默认参数	17
3.6 GRPO 主干	18
3.7 多路 reward 分解	19
3.8 Curriculum 与自适应温度	19
3.9 GRPO 默认参数	19
3.10 GRPO 一句话概括	20
3.11 研究方法插件机制	21
3.12 已实现的六个插件	21
3.13 插件 1: Verifier-Guided Repair Curriculum	21
3.14 插件 2: Clause-Aware Verifier Reward	22
3.15 插件 3: AST Anchor Interface Grounding	22
3.16 插件 4: Behavior Anchor Coverage Reward	22

3.17	插件 5: Self-Consistency Verifier Routing	23
3.18	插件 6: Uncertainty-Triggered Repair Replay	23
3.19	插件与 benchmark 的对应关系	23
3.20	TurboQuant 推理优化	23
3.21	小结	24
4	云原生集群与工程验证 (Hardware & Computing Clusters)	25
4.1	算力组织方式	25
4.2	已归档训练指标	25
4.3	训练曲线关键节点	26
4.4	8-NPU 快迭代工件	27
4.5	Gemma 主线当前进展与下一步	28
4.6	Timeboxed Pipeline	29
4.7	小结	29
5	研发架构 (软件架构)	30
5.1	Local-first 控制面	30
5.2	ai2 远端工作目录与资产分布	30
5.2.1	基座模型	30
5.2.2	数据集	30
5.2.3	微调产物 (adapter)	30
5.2.4	代码入口与控制面脚本	30
5.3	控制面组件与职责边界	32
5.4	自治研发循环	32
5.5	训练器与插件的分层设计	33
5.6	S3 Relay 同步机制	34
5.7	唯一正式报告机制	34
5.8	小结	34
6	测试结果	35
6.1	核心结果	35
6.2	评测方法与可信度	35
6.3	参考解健康度	36
6.4	详细结果分析	37
6.4.1	OmniCoder 8-NPU 评测工件	37
6.4.2	Qwen 严格量子 holdout 基线	37
6.4.3	接口层细节	37
6.4.4	评测覆盖度	38
6.5	训练指标	38

6.6	口径说明	39
6.6.1	24/26 整表数字	39
6.6.2	Delivery Gate 口径	39
6.6.3	8-NPU 快迭代工件完整度	40
6.7	Gemma 测试状态	40
6.8	小结	40
7	总结与下一步	41
7.1	当前进展总览	41
7.2	核心结论	41
7.3	下一步工作排序	41
7.4	前沿方法储备	42
7.5	业界前沿技术对标	42
7.6	展望	44
A	附录 A: 数据集与任务清单	45
A.1	A.1 数据集清单与用途	45
A.2	A.2 量子任务清单	45
A.3	A.3 软件任务清单	46
A.4	A.4 严格 holdout 的 task 划分	46
A.5	A.5 量子任务逐项说明	47
A.6	A.6 软件任务逐项说明	49
A.7	A.7 核心数据集的 category 细表	50
A.8	A.8 核心数据集的 prompt family 细表	51
B	附录 B: 研究方法插件逐项说明	52
B.1	B.1 插件接口摘要	52
B.2	B.2 Verifier-Guided Repair Curriculum	52
B.3	B.3 Clause-Aware Verifier Reward	52
B.4	B.4 AST Anchor Interface Grounding	52
B.5	B.5 Behavior Anchor Coverage Reward	52
B.6	B.6 Self-Consistency Verifier Routing	53
B.7	B.7 Uncertainty-Triggered Repair Replay	53
B.8	B.8 TurboQuant	53
C	附录 C: 工件索引与引用口径	54
C.1	C.2 数字引用矩阵	55
C.2	C.3 训练与测试 provenance 的最小引用单元	57
D	附录 D: 关键命令与复现实验	59

D.1	D.1 参考解健康度复现命令	59
D.2	D.2 严格 holdout 诚信校验命令	59
D.3	D.3 当前 8-NPU 快迭代作业的核心命令形态	59
D.4	D.4 Gemma 本地最小下一步实验	60
D.5	D.5 报告编译命令	60
E	附录 E: 训练运行逐步指标摘录	61
E.1	E.1 Qwen 8 NPU interface-prefix true20-e2 逐步指标	61
E.2	E.2 Qwen 8 NPU codefirst-semantic mix75 true20-e2 逐步指标	61
E.3	E.3 OmniCoder 9B 2 NPU continuation true20 逐步指标	62
E.4	E.4 OmniCoder 9B 8 NPU strict-quantum fastiter 逐步指标	62

摘要

本文系统性阐述了量子编码语言模型（Quantum-GPT）的后训练（Post-training）过程、数据处理管线及严格泛化测试基准。在基础监督微调（SFT）机制之上，我们引入了组相对策略优化（GRPO）强化学习架构，进而显著提升了模型在量子-经典混合环境下的编程推理和逻辑协议验证能力。

在设置严格数据隔离（Strict zero-leakage holdout）的量子计算盲测题集上，基于最新 8-NPU 分布式架构训练的 OmniCoder 适配器实现了关键的泛化突破，其不仅在通过率评估中超越基线水平（从基准 0/4 拓展至 2/4），且在语法准确性、原生接口规范以及端到端测试对齐等更全面的技术维度内验证了稳定性。本文进一步论述了包括 TurboQuant 推理压缩等前沿工程与自适应研究插件的综合框架部署。

Keywords: Quantum Coding LLM, Zero-leakage Holdout, SFT, GRPO, Post-Training Pipeline, TurboQuant

重点架构与执行概览 (Executive Summary)

本文主要面向核心研发团队及技术委员会，围绕以下五大技术链路对工程体系全周期进行剖析与能力定界：

架构与体系划分	重点度量领域
1. 研究指标总论	展示最新训练适配器在受控环境下的系统评测进度与核心迭代里程碑。
2. 评价基准与隔离	披露数据边界规范，界定监督集及多级泛化测试集的零重叠准则。
3. 后训练算法栈	透视支撑本轮突破演进的基础连续微调结构及 GRPO 测试反馈补偿网络。
4. 分布式计算堆栈	呈现本地-远端 8-NPU 训练同步的监控环境及其鲁棒性度量。
5. 体系与研发基盘	记录以云原生为主线的数据容灾机制与完全闭环自动化。

表 1: 技术文档评估模块导航 (Report Structure)

为保证验证结论的技术威信，所有输出性能指标均通过深度的隔离筛验 (Decontamination Check)。

每个正文章节均以**关键结论**开头、以**小结**收尾，便于快速扫读。所有可引用数字都附有证据编号 (如 V1、M4、B2 等)，对应附录 C 中的工件索引。

术语速查

下表列出本报告中频繁出现的专业术语及其简要解释，便于非专业读者快速理解。

1 实验体系总论 (System Overview)

1.1 研究目标

本项目的核心目标是：围绕量子算法与软件工程双域任务，构建一个**可验证、可迭代、可远端扩展**的量子编码大模型研发闭环。

本文重点论述系统的工程化实现细节、模型泛化能力的边界测定及训练策略的演进，为量子领域的大语言模型训练提供严谨的参考。

1.2 模型训练与进展概览

术语	解释
LoRA (低秩适配)	Low-Rank Adaptation, 一种高效微调技术。不修改模型全部参数, 而是在关键层注入少量可训练参数, 大幅降低训练成本。
SFT (监督微调)	Supervised Fine-Tuning, 使用标注好的问答对直接训练模型, 让模型学会特定任务的输入输出模式。
GRPO (组相对策略优化)	Group Relative Policy Optimization, 一种强化学习方法。对同一问题生成多个候选答案, 运行测试器评分, 再根据组内相对优劣更新模型策略。
Holdout (严格保留集)	专门为评估模型泛化能力而保留、训练过程中完全未见的数据集。本项目要求 example、task、prompt family 三层零重叠。
Adapter (适配器)	LoRA 训练产生的少量额外参数文件, 可以叠加在基础模型之上改变其行为, 而不需要修改原始模型权重。
NPU (神经处理单元)	Neural Processing Unit, 用于加速深度学习计算的专用硬件。本项目使用远端 8-NPU 集群进行分布式训练。
MoE (混合专家)	Mixture of Experts, 一种模型架构。模型包含多个“专家”子网络, 通过路由器 (router) 决定每个输入激活哪些专家, 兼顾模型容量与推理效率。
ESFT (专家特定微调)	Expert-Specific Fine-Tuning, 只微调 MoE 模型中被选中的少数专家, 其余专家冻结, 减少参数更新量。
Continuation training	从已有 adapter 继续训练, 而不是每次从零开始, 相当于“接力式”迭代改进。
Override (任务级通过判定)	针对严格 holdout 中每个任务的最终通过/未通过判定, 是本项目的核心 benchmark 指标。
Scorecard	一份 JSON 格式的评测记录, 记录模型在每个任务上的通过状态、错误类型与详细分析。
KV Cache	推理时模型缓存的键值对 (Key-Value), 用于加速自回归生成。TurboQuant 通过压缩 KV cache 降低长文本推理的内存开销。

表 2: 本报告核心术语速查表。

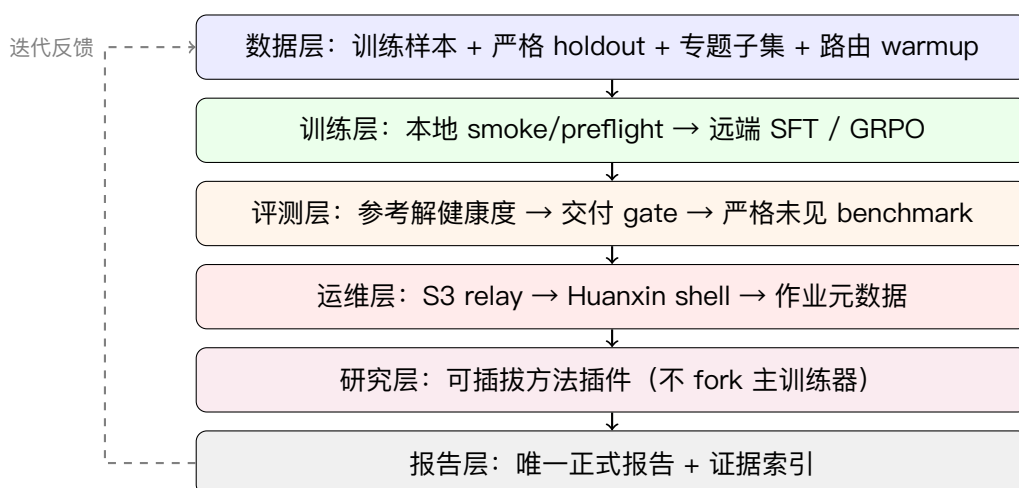


图 1: 量子编码大模型研发闭环的六层结构。每一层的输出是下一层的输入, 报告层的结论反馈指导下一轮数据与训练决策。

维度	可直接引用的数字	主证据编号
数据	混 合 holdout 1440/504; 量子 hold-out 1024/504	V1, V2
算法	GRPO 四路 reward 0.6/0.1/0.15/0.15; 6 个 plugin	A1, A2, A3
算力	4 组归档 run; 最新 8- NPU eval loss = 0.3863	M1-M4, H1
测试	参考解 26/26; over-ride 0/4 → 2/4	E2, B1, B2
Gemma	正在推进 runtime 适配 与 backend 对接	R1, G1, G2

表 3: 本轮核心可引用数字一览。详细证据见附录 C。

2 高维数据链路与评价闭环 (Data & Rigorous Evaluation Bounds)

2.1 数据设计原则

数据按三种角色组织（图 2），核心原则是：评测数据不仅要与训练数据 example 不同，还必须在 task 与 prompt family 上严格隔离。

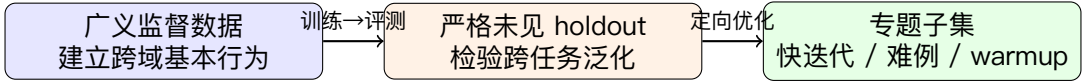


图 2: 数据的三种角色及其流转关系。

2.2 主要数据集概览

数据集	训练样本	验证样本	领域	划分策略	当前用途
template - large - v1	2208	552	量子+软件	family disjoint	最大的宽口径 SFT 基础集
mixed-holdout-v1	1440	504	量子+软件	task disjoint	混合域严格未见 holdout
quantum-large-v1	1536	504	量子	family disjoint	宽口径量子 SFT 基础集
quantum-holdout-v1	1024	504	量子	task disjoint	严格量子未见 holdout
quantum-focus-v1	252	48	量子偏置混合	随机拆分	快速小样本量子迭代
quantum-hard-v1	121	19	量子	task subset split	难例聚焦与压力测试
paper - router / messages	42	5	论文路由消息	消息拆分	Gemma 路由 warmup 前置数据

表 4: 当前主要数据集的规模与角色。

图 3 说明仓库已摆脱“小样本玩具数据”的状态。数据规模是“在有限任务库约束下的强纪律”，而非大规模公开 benchmark。

2.3 混合域严格 holdout

omnicoder-generalization-holdout-v1 的核心性质见表 5，三层零重叠保证了评测的严格性。

检查项	训练侧	验证侧	结果解释
样本数	1440	504	验证集规模达到 500+，满足领导口径要求
task 数	15	8	训练/验证任务集合完全分离
prompt family	6	4	训练/验证提示风格也分离
领域分布	量子 768，软件 672	量子 252，软件 252	验证集在量子/软件上完全平衡
重叠校验	example_id、task_id、prompt_family overlap 均为 0		

表 5: 混合域严格 holdout 的可信度要点。

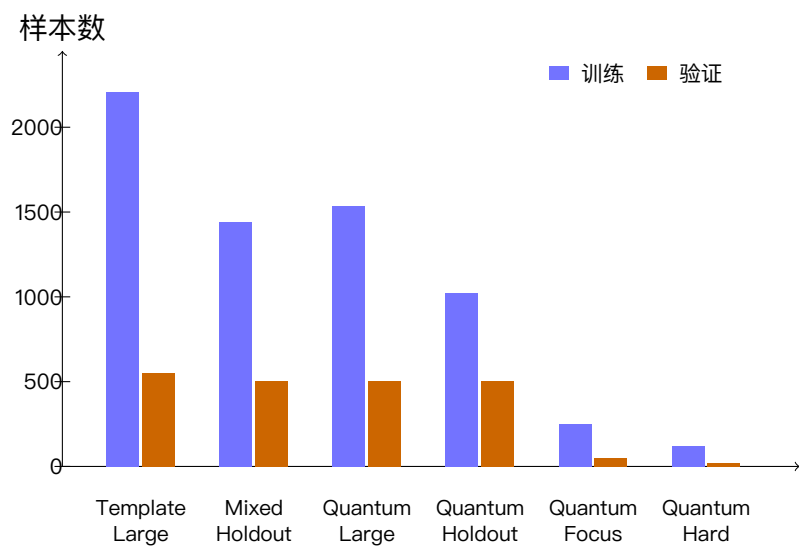


图 3: 六个主要监督数据集的训练/验证规模对比。

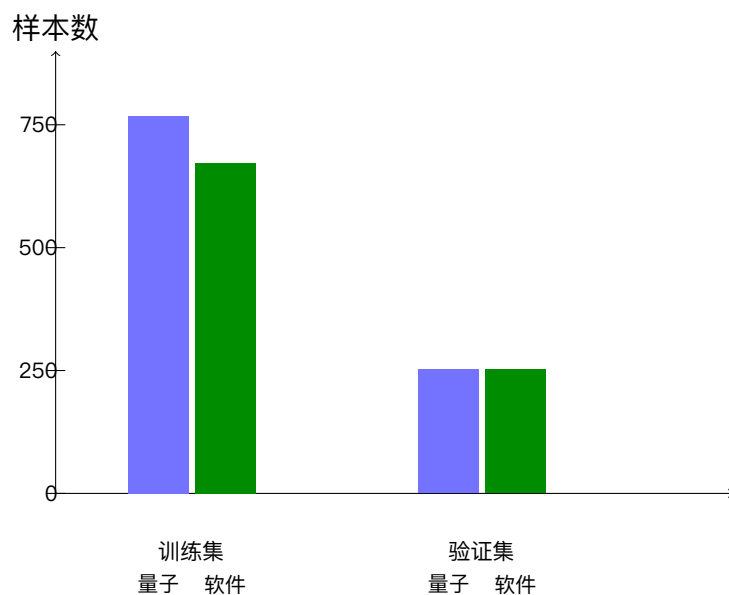


图 4: 混合域严格 holdout 的训练/验证领域分布。

混合域 holdout 的验证集在量子/软件上严格平衡（图 4），使得“量子提升是否伤害软件”可直接追踪。

2.4 量子严格 holdout

omnicoder-quantum-generalization-holdout-v1 比混合 holdout 更激进：训练仅 8 个量子 task，验证仅 4 个量子 task，且该部分验证任务集中考察当前模型的薄弱环节（表 6）。

检查项	训练侧	验证侧	结果解释
样本数	1024	504	训练样本按 8 个任务平均分布，验证样本按 4 个任务平均分布
task 数	8	4	task 完全不重叠
prompt family	6	4	prompt family 也完全不重叠
领域分布	全部量子	全部量子	专门服务量子未见任务泛化
验证任务	gate _ alias _ normalization、phase _ estimation _ circuit、qaoa _ maxcut、superdense_coding		

表 6: 量子严格 holdout 的核心性质。

上述 4 项验证任务覆盖了四类关键能力：API 正规化、经典-量子算法实现（2 个）、协议型推理。当 Qwen 在此 benchmark 上 0/4 时，可合理解读为“严格未见量子任务仍明显困难”。

2.5 prompt family 划分

两个 holdout 不仅 task disjoint，prompt family（提示风格族）也完全分离。训练侧使用 6 种风格（如 behavior_first、implementation_ticket），验证侧使用 4 种不同风格（如 acceptance_gate、spec_translation）。这表明模型必须在新任务背景和新说话方式下完成迁移。

2.6 样本配额

两个严格 holdout 在 task 粒度上接近等额配给（表 7），避免单个大类任务的超大配额掩盖其他任务的表现起伏。

数据集	聚合口径	类别数	单类样本规模	说明
混合 holdout 训练侧	按 task 计	15	每 task 96	各 task 等额，避免训练侧被单一题型主导
混合 holdout 验证侧	按 task 计	8	每 task 63	每个验证 task 权重相同，便于逐题追踪
量子 holdout 训练侧	按 task 计	8	每 task 128	更激进地把量子训练 task 做成等额配额
量子 holdout 验证侧	按 task 计	4	每 task 126	四个关键未见量子任务权重完全均衡

表 7: 两个严格 holdout 在 task 粒度上的样本配额。

值得一提的是，当前 holdout 不只是“量子/软件各占一半”或“样本总数够多”，而是在任务类别上刻意覆盖了不同难度信号。以混合 holdout 为例，训练侧包含 algorithm_

prompt family	混合 holdout 训练/验证	量子 holdout 训练/验证	解释
family 数量	6 / 4	6 / 4	两个严格数据集都坚持 train/test 风格分离
训练侧配额	240/240/240/240/240/240	176/176/168/168/168/168	混合 holdout 完全等额, 量子 holdout 接近等额
验证侧配额	128/128/128/120	128/128/124/124	验证风格分布接近均匀, 不依赖单一话语模板

表 8: prompt family 的数量与配额。

implementation、debug_repair、protocol_logic、test_writing、stateful_logic 等不同类型；验证侧则刻意换成 circuit_construction、api_normalization、multifile_repair、design_pattern 等另一批类型。也就是说，当前验证并不是在熟悉类型内换个题面，而是有意识地让模型面对“任务类型 + 话语风格 + 具体实现目标”三重迁移。

类别聚合	混合 holdout 训练侧样本数	混合 holdout 验证侧样本数	解读
algorithm implementation	288	0	训练侧大量覆盖算法实现，但验证侧换成别的量子能力类型
debug_repair	96	126	修复类能力在训练和验证两侧都保留，但 task 完全不同
stateful / multifile / design pattern	96	189	软件验证侧偏向状态管理、多文件与设计模式任务
API / interface 类	96	63	验证侧单列 API 正规化，利于直接观察接口对齐情况

表 9: 混合 holdout 中最重要的类别迁移信号。

这一点决定了当前 benchmark 的**解释边界**。它可以支持“模型在严格未见 task 上是否开始具备迁移能力”的讨论，但不能支持“模型已经在开放世界量子编程中普遍泛化”的夸张表述。换言之，当前数据纪律足够强，可以支撑研究结论；但数据规模仍然是封闭任务集上的强纪律，而不是开放宇宙中的大规模公开 benchmark。

2.7 Category 与 Prompt Family 细分

如果只报告训练/验证总样本数，仍然无法判断数据是否过度集中在少数任务形态上。当前最值得在正式报告中展开的，是四个核心数据集的 category 与 prompt family 细分：omnicoder-template-large-v1、omnicoder-quantum-large-v1、omnicoder-generalization-holdout-v1、omnicoder-quantum-generalization-holdout-v1。它们共同描述了仓库在“宽口径监督”与“严格未见验证”之间的分工。

综上所述，宽口径数据的主峰恰恰是项目最需要的能力类别；holdout 验证集在 category、prompt family、task 三个层面同时制造迁移压力。

数据集	train domain 分布	eval domain 分布	可直接解释的结构特征
omnicoder-template-large-v1	量子 1152, 软件 1056	量子 288, 软件 264	最大的宽口径混合监督集, 量子/软件接近平衡, 适合作为基础 SFT 底座
omnicoder-quantum-large-v1	量子 1536	量子 504	纯量子宽口径监督集, 适合在不引入软件域噪声的前提下加强量子能力
omnicoder-generalization-holdout-v1	- 量子 768, 软件 672	量子 252, 软件 252	严格 mixed holdout, 验证集在两个 domain 上完全平衡
omnicoder-quantum-generalization-holdout-v1	- 量子 1024	量子 504	strict quantum-only holdout, 用于最诚实地测量量子未见任务泛化

表 10: 四个核心监督数据集的 domain 分布。

数据集	train / eval category 结构摘要
omnicoder-template-large-v1	train 侧 debug_repair=288、algorithm_implementation=288、stateful_logic=288 为三个最大簇, 其余 14 类均各 96; eval 侧上述三类均为 72, 其余 14 类均各 24。它是真正意义上的“宽口径但仍可控”的基础监督集。
omnicoder-quantum-large-v1	train 侧 debug_repair=384、algorithm_implementation=384, 其余 circuit_construction、circuit_optimization、api_normalization、algorithm_reasoning、reasoning、protocol_logic 均各 128; eval 侧相应为 126 与 42。它明确偏向“量子修复 + 量子算法实现”双主峰。
omnicoder-generalization-holdout-v1	- train 侧只有 algorithm_implementation=288 高于其他类, 其余 12 类各 96; - eval 侧由 debug_repair=126 与 stateful_logic=126 主导, 其余 circuit_construction、api_normalization、multifile_repair、design_pattern 各 63。
omnicoder-quantum-generalization-holdout-v1	- train 侧 debug_repair=384 为最高, circuit_construction、circuit_optimization、algorithm_reasoning、protocol_logic、algorithm_implementation 各 128; eval 侧则集中为 algorithm_implementation=252、api_normalization=126、reasoning=126。

表 11: 四个核心监督数据集的 category 结构摘要。

数据集	train family 配额	eval family 配额	解释
omnicoder-template-large-v1	六类均为 368	四类均为 138	当前最规则的 family-disjoint 大集合之一, 适合作为基础 prompt 风格底座
omnicoder-quantum-large-v1	252-264	120-132	量子宽口径集近似均衡
omnicoder-generalization-holdout-v1	- 六类均为 240 -	128/128/ 128/120	mixed holdout 的训练风格最均匀, 便于把验证压力集中在 task 与 domain
omnicoder-quantum-generalization-holdout-v1	- 176/176/168/ 168/168/168 -	128/128/ 124/124	strict quantum holdout 在风格上仍保持较高均衡, 因此 0/4 结果不能简单归咎于 prompt style 偏差

表 12: 四个核心监督数据集的 prompt family 分布。

2.8 快迭代子集与难例子集

仓库还保留两个小规模专题集，适合局部优化与方法消融，但不承担最终领导口径：

数据集	selected	train/eval	domain 分布	当前最适合的用途
omnicoder quantum-focus-v1	- 300	252/48	selected 阶段量子 240, 软件 60; train 阶段量子 202, 软件 50; eval 阶段量子 38, 软件 10	当需要保持“量子优先，但不彻底丢掉软件信号”时，用于 2 NPU 或短时长 continuation 快试验
omnicoder quantum-hard-v1	- 140	121/19	全部量子；7 个 hard task 组成，适合做 targeted stress test	当需要验证某个方法是否真的提升 hardest quantum slice 时，用于快速 hard-subset 验证

表 13: 两个快迭代专题子集的规模与定位。

数据集	任务分布摘要
omnicoder quantum-focus-v1	- selected_summary 显示 12 个量子任务各 20 条，说明初选阶段是均匀量子覆盖；train/eval 再随机拆分后，qft_phase_pattern=19、stabilizer_tableau_update_repair=18、teleportation_corrections=18、vqe_energy_minimization=18 等成为 train 侧高频任务。
omnicoder quantum-hard-v1	- 由 delivery_pass1_quantum_hard_v1.txt 指定 7 个 hard quantum task，各 task 在 selected 阶段均为 20 条；train 阶段最高为 phase_estimation_circuit=19，最低为 circuit_phase_repair=15；eval 仅 19 条，因此更适合做“局部难例趋势”而不是做 headline。

表 14: 两个专题子集的任务组成摘要。

2.9 Gemma router warmup 数据

Gemma 路由 warmup 的消息数据规模小但来源可追踪（表 15）。每条记录包含 source、chunk_index、char_start/end，可回溯到具体论文与字符区间。

source	train 条数	valid 条数	说明
paper / novel _ rl _ algorithms.tex	37	5	当前 paper-router warmup 的主来源，训练与验证都覆盖到该文件
paper/quantum-coding-llm-rnd.tex	5	0	当前仅出现在训练侧，属于第二来源文件

表 15: Gemma paper-router warmup 消息数据的 source 分布。

当前数据偏向单一来源（novel_rl_algorithms.tex 占主体），适合承担”路由器轻量预热”而非”最终专家蒸馏”。

2.10 Gemma 路由 Warmup 数据边界

当前状态：消息级 warmup 42/5 条存在，但 PDF 级记录文件行数为 0（表 16）。因此这是消息级预热，不是完成的 PDF 论文对齐语料。

工件	行数	解释
messages / quantum _ paper_router_warmup_ messages_train.jsonl	42	当前可直接用于消息路由 warmup 的训练数据
messages / quantum _ paper_router_warmup_ messages_valid.jsonl	5	当前可直接用于消息路由 warmup 的验证数据
pdf_records/quantum_ paper_router_warmup_ all.jsonl	0	文件存在，但尚未形成可引用 PDF 记录
pdf_records/quantum_ paper_router_warmup_ train.jsonl	0	文件存在，但尚未形成可引用 PDF 记录

表 16: Gemma 路由 warmup 数据的真实当前状态。

必须明确界定该能力边界，因为一旦将“已有 42/5 消息 warmup”误写成“已有完善 PDF 论文路由数据”，就会直接把研究状态夸大。当前最合理的策略不是回避这个问题，而是把它写成数据层的下一步建设项。

2.11 实验纪律

数据结构直接导出三条纪律：(1) 声称对严格 holdout 有效的实验必须说明数据子集与是否污染验证 task；(2) 仅在 focus/hard 子集上改善的不能作为主结论；(3) Gemma warmup 在 PDF 记录补齐前只能算”数据准备中”。

2.12 小结

当前数据层具备三点成熟度：宽口径监督数据可支撑 SFT；严格 holdout 可支撑泛化结论；Gemma warmup 专题数据存在但仍需补齐 PDF 级工件。后续实验不应把”数据太乱”作为借口。

3 模型后训练架构：预调与反馈纠正 (Post-training Methods)

3.1 算法栈总览

当前算法栈如图 5 所示，可用”主干稳定、方法可插拔”概括。

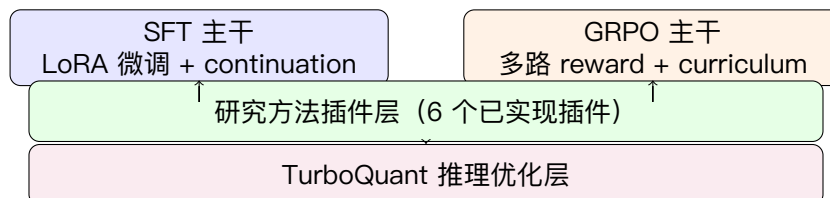


图 5: 四层算法栈结构。SFT 与 GRPO 是稳定主干，插件层注入研究方法，TurboQuant 服务推理优化。

层次	代表实现	当前状态	主要作用
监督训练主干	training/qwen_sft_peft.py	已实现并长期使用	将聊天式 JSONL 数据转成 LoRA 微调任务
强化学习主干	training / grpo_trainer.py	已实现并可发起远端作业	按组采样候选、运行测试器、依据 reward 相对优势更新策略
研究方法层	training/research_plugins.py + 各 parser plugin	已实现并接入 SFT/GRPO	在不 fork trainer 的前提下插入方法研究
推理优化层	training/turboquant.py	/ 实验性实现	降低长上下文推理时 KV cache 压力

表 17: 当前算法栈的四层结构。

3.2 SFT 主干

training/qwen_sft_peft.py 的功能远不止“读 JSONL 然后训 LoRA”。现有实现至少包含七类能力：

聊天式消息数据读取与文本预处理；

train_on_completions_only 的 completion-only mask；

adapter-init continuation training；

LoRA 目标模块自动发现与手动覆盖；

基于正则表达式的选择性参数训练与冻结；

research method 对 SFT record 的增广；

runtime 与 backend preflight。

这表明当前 SFT 主线已经可以同时服务三类看似差异很大的实验：

标准 Qwen/OmniCoder 文本路径 LoRA 微调；

继续训练已有 adapter，而不是每轮从头开始；

针对 Gemma 路由器或 gate 的局部参数 warmup。

以 continuation training 为例，--adapter-init 使得快迭代不必从较弱底座重复起步，而

是可以从已验证过的 adapter 继续向前推进。对于算力昂贵的远端环境，这一点至关重要。它把“每轮重新热身”的低效率行为，替换成“以最好已知权重为起点”的研究迭代。

3.3 选择性训练控制

当前 SFT 脚本提供了两组至关重要、且容易被忽略的能力：`--target-module-regex` 与 `--trainable-param-regex/--freeze-param-regex`。这两类控制的意义在于，它们让模型微调从“整个 attention/MLP 层一刀切插 LoRA”扩展为“精确选择某类模块或参数”。

对于 Gemma 4 26B-A4B 这样的条件生成/MoE 目标，这种能力尤其重要。因为当前最现实的周末 demo 主线，并不是宣称“已经能完整训好 Gemma”，而是先做更小范围、更容易解释、更便宜的 router/gate warmup。对应的策略不是暴力改整个模型，而是通过正则限制只让指定模块变为可训练。

3.4 Gemma MoE 微调策略

这里必须明确一点：在当前仓库的已验证 runtime/backend 条件下，不应把 Gemma 4 26B-A4B 按普通 dense causal LM 的默认 LoRA 路径直接作为可执行主线。从本地快照与控制面代码看，该目标按 MoE/专家调优思路被规划；当前仓库中的 `training/inspect_moe_target_modules.py` 也把推荐策略预设成 `router_warmup_then_frequency_guided_esft`。因此，后续对 Gemma 的正式 finetuning，当前更稳妥的规划是“先 router/gate，后 expert；先拿 routing 证据，再谈 expert 选择”。

但这仍然是计划中的方法，不是当前已经完成的结果。当前仓库已经具备的是：MoE 模块发现、regex 级 selective training 控制、router warmup 命令生成；当前还没有完成的是：Gemma 4 conditional-generation backend、packed expert tensor 的真 expert-slice 训练、以及两阶段 adapter 的无缝衔接。

MoE 微调环节	当前状态	报告应如何表述
router/gate 模块发现	已具备本地扫描脚本	可以写“仓库已具备 MoE 结构扫描脚本与 router warmup 规划能力”，然而否完成发现仍以实际扫描工件为准
router-only warmup 命令生成	已具备命令模板	可以写“已准备 router/gate 定向 warmup 方案”，不能写“已完成 Gemma router 训练”
frequency-guided ESFT	仅有方法规划	只能写为下一步计划，不能写成已验证收益
packed expert tensor 的真 expert 选择	未完成	当前还不能声称已经做到按 top-K expert 的精确微调
Gemma conditional-generation SFT backend	未完成	runtime 适配完成后的下一步工作

表 18: Gemma 4 26B-A4B 的 MoE-aware 微调边界。

3.5 SFT 默认参数

为了避免报告把训练器描述成抽象黑盒，本节把当前 `training/qwen_sft_peft.py` 中几个最关键的默认参数直接写清楚。当前默认配置为：

```

max_length=2048;

per_device_batch_size=1;

gradient_accumulation_steps=8;

learning_rate=2e-4;

num_epochs=1;

max_steps=10;

eval_steps=5;

log_steps=1;

lora_rank=16, lora_alpha=32, lora_dropout=0.05;

completion-only 训练默认关闭，需要显式传入 --train-on-completions-only。

```

参数	默认值	研究解释
--adapter-init	无	只有在 continuation 场景下显式打开，避免默认把所有实验都变成“接着前一轮训”
--target-module-regex	无	默认仍以 auto-discovery 为主；只有 router-only 或 expert-only 微调时才缩窄目标模块
--trainable-param-regex	无	默认不做更细粒度参数过滤；研究者需主动声明 selective training 规则
--freeze-param-regex	无	
--train-on-completions-only	false	表示默认 SFT 仍可以学习完整对话样式，而不是自动假设所有任务都适合 completion-only
--max-steps	10	这也是为什么很多 smoke/fastiter 结果应该被表述为“短程稳定性验证”而不是“大预算训练”

表 19: SFT 主干中最关键的默认参数及其解释。

从研究角度看，这套默认值透露出当前训练器的真实定位：它首先是一个**高可控、适合做 bounded experiment 的工程平台**，然后才是一个追求长程极限训练的脚本。也正因为如此，当前很多可信结论来自“20 step 真实工件”而不是“上百 step 长训猜想”。

3.6 GRPO 主干

training/grpo_trainer.py 的引入意味着算法栈已经越过了“只能做 SFT”的阶段。当前 GRPO 训练器具备以下核心机制：

- 对每个任务 prompt 一次采样多个候选；
- 将候选代码交给 task harness 执行，而不是只看语言模型分数；
- 将 pass/fail 进一步分解为多路 reward；
- 用组内相对优势而不是绝对 reward 直接更新策略；
- 用 curriculum 与 adaptive temperature 避免低信号无效更新。

在数学上，现有实现最重要的两个对象是总 reward 与相对优势。总 reward 的默认组合为

$$r = 0.6 r_{\text{pass}} + 0.1 r_{\text{syntax}} + 0.15 r_{\text{interface}} + 0.15 r_{\text{verifier}} + \lambda_{\text{brevity}} r_{\text{brevity}}.$$

其中 brevity reward 默认权重为 0，但机制本身已经存在，用于某些“所有候选都同样错误但长短差异显著”的边角情形。

相对优势的基本思想则可以表述为

$$A_i = \frac{r_i - \bar{r}}{\max(\sigma_r, \epsilon)},$$

其中同一组候选的平均 reward $\bar{r}$ 与标准差 σ_r 决定了这个 candidate 相对组内其他 candidate 的优势。这样做的直观意义是：即使一整组候选都没有完全通过，只要其中某个候选“更接近正确”，模型仍能从这个相对差异中得到学习信号。

3.7 多路 reward 分解

代码生成任务尤其是量子编码任务，常常出现“接口存在细微偏差、逻辑存在细微偏差、只差一个边界条件”的 near-miss 状态。如果 reward 只有 pass/fail，训练信号就会极度稀疏。当前 GRPO 训练器因此明确拆分成四路默认 reward：

pass reward：最终测试是否通过；

syntax reward：代码是否至少满足基本可解析性；

interface reward：导出函数名、签名、接口形状是否对齐；

verifier reward：来自验证器或测试器细粒度反馈的部分信用。

这背后体现的并不是“把分数凑复杂”，而是对真实错误分布的正视。对于量子编码任务，很多未通过不是“完全不会”，而是：

大方向对，但函数名错；

量子态或门序列结构对，但符号大小写错；

主要推理路径对，但边界 case 或返回值格式错。

若无法将此类边缘失效（near-miss）转化为可学习的有效强化信号，RL 就很容易停在“全错即零分”的低效状态。

3.8 Curriculum 与自适应温度

当前 GRPO 训练器不是在静态均匀采样任务。其策略由 `TaskCurriculum` 与 `AdaptiveTemperatureState` 共同调节：

`task curriculum` 依据每个任务的历史 EMA reward、出现次数、难度与不确定性动态调权；

`quantum priority` 可以对量子任务施加更高采样权重；

当某一步 reward 方差过低，训练器会选择跳过本步更新，而不是硬做无效梯度；

连续跳过会触发采样温度上调，以增大候选多样性。

这套机制的意义在于，本方案将“训练信号的优质性”视作核心，而非机械地关注迭代轮次。对于当前这样的小任务集与高成本远端环境，这一点直接决定了 RL 是否值得跑。

3.9 GRPO 默认参数

当前 `training/grpo_trainer.py` 的默认参数也值得在正式报告中单列，因为它们直接决定了“当前 RL 主线是在探索什么样的训练 regime”。根据本地源码，当前默认值包括：

```
group_size=8;
```

```

grpo_steps=100;

lr=1e-5;

kl_coeff=0.05;

temperature=0.8;

adaptive_temp_step=0.15, adaptive_temp_max=1.4;

max_new_tokens=2048, max_seq_length=4096;

reward_pass_weight=0.6, reward_syntax_weight=0.1, reward_interface_weight=0.15, reward_verifier_weight=0.15;

reward_brevity_weight=0.0;

min_reward_std=0.05, curriculum_ema_decay=0.9, curriculum_min_weight=0.05, curriculum_uncertainty_bonus=0.35, quantum_priority=1.5。

```

参数簇	默认值	报告中应如何解释
组采样	group_size=8	每个 prompt 默认生成 8 个候选，再在组内做相对优势比较，说明当前 RL 不依赖单一样本的脆弱 reward
温度控制	0.8 基础温度，+0.15 逐次放大，上限 1.4	当 reward 信号过平时，不是盲目继续更新，而是先提高采样多样性
reward 结构	0.6/0.1/0.15/0.15	pass 仍是主导信号，但 interface 与 verifier 已经被纳入同等重要的辅信号
跳步阈值	min_reward_std=0.05	当 total reward 和 component reward 同时过平时，训练器会跳过该步，避免无效更新
curriculum	ema = 0.9, min_weight = 0.05, uncertainty_bonus = 0.35, quantum_priority=1.5	当前 RL 明显偏向量子任务，并显式奖励高不确定任务，而不是静态均匀采样

表 20: GRPO 主干的默认参数面板。

这张表之所以重要，是因为它让“当前 RL 在干什么”从笼统叙述变成了可审阅的参数事实。对于组会讨论而言，这表明我们可以把争论具体化为：是应该继续增大 group size、调高 verifier 权重、还是降低 skip 阈值，而不是泛泛争论“RL 有没有用”。

3.10 GRPO 一句话概括

如果要在组会里用一句话概括当前 RL 主线，最准确的表述是：目前采用 GRPO，它不是对单个答案做 0/1 打分，而是对同一个任务一次采样 8 个候选，把候选交给测试 harness 执行，再用 pass、syntax、interface、verifier 四路 reward 的加权结果做组内相对优势更新。

这句话里每个部分都对应一个明确实现事实：“GRPO”对应 training/grpo_trainer.py 的组相对策略更新主干；“一次采样 8 个候选”对应默认 group_size=8；“交给测试 harness 执行”意味着 reward 来源不是语言模型自打分，而是任务级测试反馈；“四路 reward”对应当前默认的 0.6/0.1/0.15/0.15；“组内相对优势更新”意味着训练信号来自同组候选之间的相对好坏，而不是某个固定阈值以上才有梯度。

从研究解释上看，这种设计专门服务当前量子编码任务的 error profile：大量失败并不是“完全不会”，而是接口缺一项、命名存在细微偏差、协议差一步、边界 case 缺失一个重要分支。因此，当前 RL 不是在追求一个抽象的“更高奖励”，而是在把这些 near-miss 变成可学习的细粒度信号。对于本项目来说，reward 设计本身就是方法设计的一部分，而不是训练器外面的附属超参数。

3.11 研究方法插件机制

training/research_plugins.py 定义了一套相当克制、但非常实用的插件接口。每个研究方法可以在以下四个关键点注入：

augment_sft_record：在 SFT 阶段修改样本；
augment_grpo_prompt：在 GRPO 阶段修改 prompt；
adjust_task_weight：在 curriculum 层调整任务采样权重；
adjust_reward_breakdown：在 reward 层调整分解结果。

这样的接口设计有三个明显好处：

新方法可以独立存在于 research/papers/<method>/code/plugin.py，不会污染主训练器；

SFT 与 GRPO 共用同一方法注册方式，便于做消融；

报告中可以明确区分“主干能力”和“研究插件能力”，避免把所有改动混成一个黑盒。

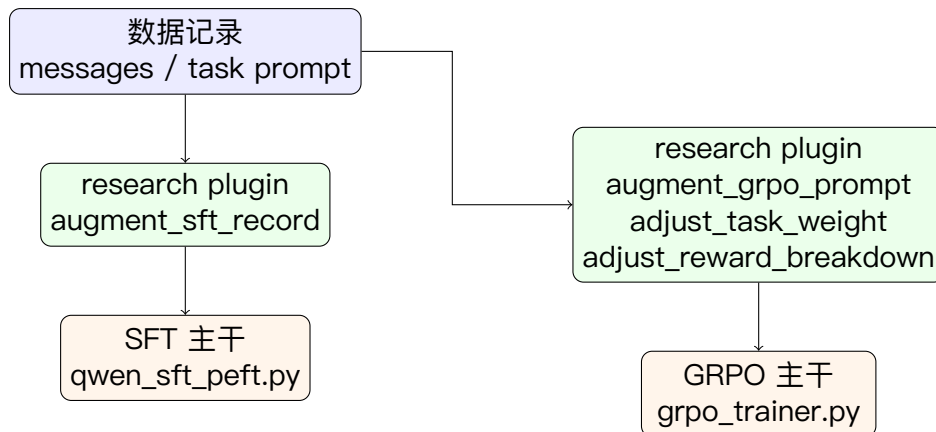


图 6: research plugin 在 SFT 与 GRPO 两条主干中的挂载位置。

3.12 已实现的六个插件

当前已有 6 个真正以插件代码形态存在的方法，它们都位于 research/papers/*/code/plugin.py：

3.13 插件 1: Verifier-Guided Repair Curriculum

这个方法的核心不是“让模型自由发挥”，而是将问题先转化为“最小必要修复”。对很多代码任务尤其是修复类任务而言，最危险的行为不是小错，而是模型在修正一个局部问题时顺手重写了整个实现，导致接口或边界行为一起漂移。

因此，这个插件在 SFT 侧会强调 repair-first 的系统提示，在 GRPO 侧则会把 prompt 重写为“保持接口、优先补齐待提升点、避免额外改动”的风格。对于当前量子任务集，这个方

方法	主要作用阶段	当前状态	方法要点
verifier_guided_repair_curriculum	SFT + GRPO	已实现	强调最小修复、接口保持、near-miss 修复优先级
clause_aware_verifier_reward	GRPO	已实现	将 verifier 反馈拆成条款级部分信用
ast_anchor_interface_grounding	SFT + GRPO	已实现	先把接口和导出形状学对，再追求复杂行为
behavior_anchor_coverage_reward	GRPO	已实现，偏向探索	用低成本行为锚点补充稀疏 reward
self_consistency_verifier_routing	SFT + GRPO	已实现	偏向 verifier 一致的候选与思路路由
uncertainty_triggered_repair_replay	SFT + GRPO	已实现	对高不确定、高难度、near-miss 样本做 replay 与修复强化

表 21: 当前已经代码化的 research plugin。

向尤其合理，因为很多任务本质上就是“修正已有门序列、测量映射或返回结构”。

3.14 插件 2: Clause-Aware Verifier Reward

该方法承认一个事实：测试器返回的信息不应该只被压缩成 0/1。很多未通过的细节本身就包含了条款级监督。例如“大小写不正确”“缺失某个函数”“某个消息映射错误”。将此类反馈将被转化为奖赏系统中的局部信用，从而为强化学习提供更为密集和优化信号。

在现有实现里，这种思路被编码为 verifier reward 的保守增益，而不是取代原始 pass reward。这样做可以防止模型专门去优化“看起来像对”的局部结构，却忽视了最终通过率。

3.15 插件 3: AST Anchor Interface Grounding

量子编码任务和软件工程任务有一个共性：接口形状错了，后续行为往往再好也无法通过。当前插件通过候选代码的导出符号、函数签名与接口线索，将“先把 API 说对”作为一等目标。对严格 holdout 中的 phase_estimation_circuit 与 qaoa_maxcut 这类任务而言，这尤其关键，因为当前 Qwen 基线的表现恰恰是函数缺失或函数名不匹配。

3.16 插件 4: Behavior Anchor Coverage Reward

这是当前方法层中最“轻量”也最容易被快速放弃的一个方向。本方法从测试用例或行为指令中主动提取核心锚点 token，如关键函数名、关键术语、关键动作，再用它们是否被候选代码覆盖来生成廉价 reward。它的价值不在于单独决定好坏，而在于为“所有候选都没有真正通过”的情况下提供一点结构化差异。

这类方法之所以值得保留，是因为它几乎不增加远端训练成本；之所以必须写明“偏探索”，是因为它一旦不能显著提高信号方差，就应当被快速停用。

3.17 插件 5: Self-Consistency Verifier Routing

这一方向并不追求让模型输出多份答案，而是把“更像 verifier 期望的思路”作为路由偏好写回训练过程。对编程类任务而言，代码最终是否能被测试器接受，比“解释得是否漂亮”更重要；该插件因此强调候选间的一致性、接口正确性与 verifier 对齐。

3.18 插件 6: Uncertainty-Triggered Repair Replay

当前量子任务集规模仍然有限，因此每个难任务都很宝贵。该插件试图用 difficulty、category、prompt family、behavior hints、接口复杂度等信号估计任务 hardness，并在模型表现不稳定时进行更有针对性的 replay。与简单的“按未通过次数重采样”相比，这种做法更接近研究意义上的不确定性驱动训练。

3.19 插件与 benchmark 的对应关系

下表基于严格量子 holdout 的评测结果与现有 plugin 机制，推导出下一步方法优先级矩阵，说明各研究插件与具体提升方向的对应关系。

目标任务	当前状态	最直接对应的方法	预期改善方向
quantum gate _ alias normalization	大小写正规化待提升	AST Anchor + clause-aware reward	接口与 token 规范收紧
quantum _ phase estimation_circuit	函数接口待补全	AST Anchor + re-pair curriculum	先学会导出正确函数名与接口
quantum _ qaoa maxcut	关键函数待补全	AST Anchor + re-pair curriculum	将“缺函数”推进到完整可调用
quantum superdense_coding	协议 round-trip 待完善	verifier routing + uncertainty replay	区分部分正确与完整闭环

表 22: 基于 benchmark 分析推导出的 plugin 优先级矩阵。

这张表展示了各研究插件的具体应用方向：当前需要提升的四个任务中，两个涉及函数接口补全，一个涉及 API 规范化，一个涉及协议逻辑闭环，它们分别对应接口锚定、最小修复、条款级 reward 与不确定性 replay 这几类方法。方法栈的设计直接围绕具体提升方向展开。

3.20 TurboQuant 推理优化

research/papers/turboquant/paper.md 与 training/turboquant.py 指向的是一条推理时 KV cache 压缩的研究方向，而不是训练算法。现有实现已经有完整的配置对象与量化/反量化路径，目标是降低长上下文推理时的显存或 NPU 内存压力。它适合接入 scripts/serve_openai_chat_adapter.py 或 scripts/run_hf_pass1_eval.py 这样的推理路径。

把 TurboQuant 写进本报告是因为它是方法栈的重要组成部分。需要说明的是：TurboQuant 属于推理优化方向的实验性实现，服务于长上下文推理效率，与训练主线分开叙述。

3.21 小结

当前算法层已经具备以下研究价值：

- SFT 主线足够稳定，能够支撑持续数据迭代；

- GRPO 主线已经成型，不再受限于“只能模仿训练集”；

- 方法研究已经被工程化为插件，而不是一次性分支；

- TurboQuant 提供了算力效率方向，为后续长上下文推理实验预留了入口。

因此，算法层当前真正稀缺的不是“再想一个新 idea”，而是把已有主干和插件在严格 holdout 上持续做出可信实验。

4 云原生集群与工程验证 (Hardware & Computing Clusters)

4.1 算力组织方式

算力分两条线运行（图 7），避免因等待 Gemma 而让整体迭代停滞：

主线： Gemma 4 26B-A4B，目标是更有研究意义的模型转型与路由 warmup；

快线： OmniCoder 9B continuation，用于在 ai2 上保持高频迭代。

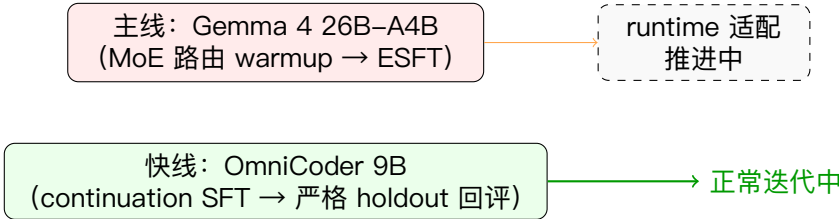


图 7：双线并行策略。快线保持高频迭代，主线正在推进 runtime 适配。

4.2 已归档训练指标

当前至少有四组训练工件可以直接进入正式报告，它们都带有独立的 `metrics.json`：

运行目录	模型	NPU 数	train/eval	step 数	最终 eval loss	解释
outputs / interface - prefix - semantic - v4 - 8npu - true20 - e2 - 20260326T1627CST	Qwen2.5- 1.5B	8	160/32	20	0.6986	8 NPU 文本 SFT 路径可稳定完成
outputs / codefirst - semantic - mix75 - 8npu - true20 - e2 - 20260326T2118CST	Qwen2.5- 1.5B	8	160/32	20	0.7353	改 prompt/数据混合方式后 略差于 interface- prefix
outputs / interface - prefix-omnicoder9b- semantic - v4 - 2npu - true20 - 20260329T2219CST	OmniCoder- 9B	2	160/32	20	0.3727	2 NPU continuation 路径表现显著 更强
outputs omnicoder9b quantum generalization sft-8npu-fastiter- 20260409T1451CST	/ OmniCoder- 9B	8	1024/504	12	0.3863	严格量子 holdout 上的 8 NPU 快迭代已 形成归档工件

表 23：已经稳定归档、可直接对外引用的训练指标。

图 8 不应被误读为“不同模型绝对公平比较”。这四组实验的数据规模、初始化状态与训练预算并不完全相同。它真正证明的是：当前训练堆栈在 8 NPU 和 2 NPU 两种分布式设置下都已经可以形成稳定可归档工件，而且最新 8 NPU run 已经把严格量子 holdout 接入了真实归档链路。

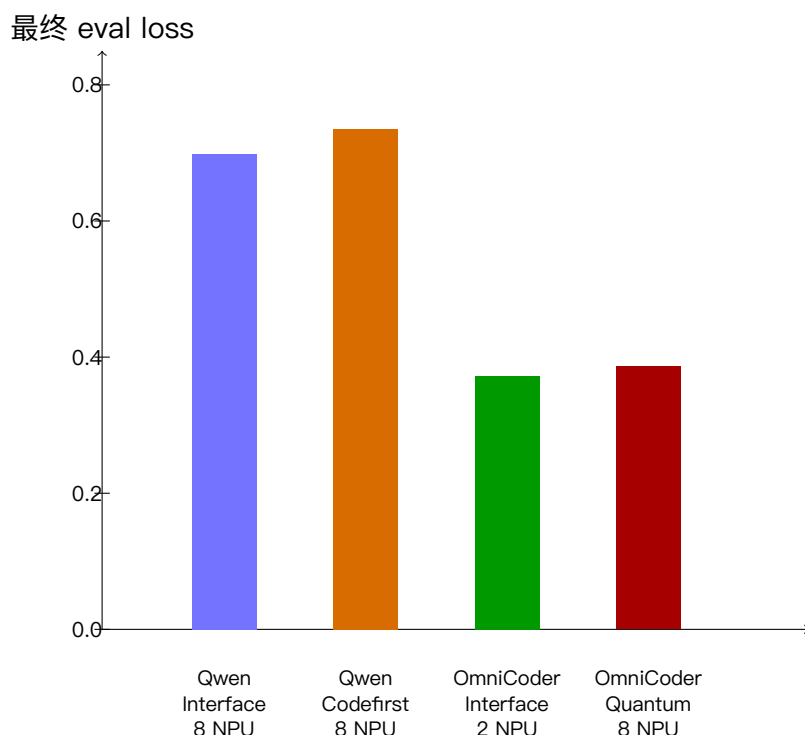


图 8: 四组稳定归档训练工件的最终 eval loss 对比。

4.3 训练曲线关键节点

仅评估最终的损失值 (Loss) 尚显不足。对快迭代而言，训练曲线的中途检查点同样关键，因为它决定了我们是应该延长 step、提前停训，还是换底座。当前四组归档工件都保留了 step 级指标，因此需要对比其在训练中期与收敛时的差异。

工件编号	step 1 train loss	首次 eval 点	最终 eval 点	变化量	解释
M1	3.2799	0.8348	0.6986	-0.1362	Qwen interface–prefix 继续下降，说明 20 step 没有过早饱和和 code–first 也下降，但整体慢于 M1
M2	3.2799	0.8642	0.7353	-0.1288	
M3	1.0787	0.4477	0.3727	-0.0750	OmniCoder 起点更强，中后段继续细化而非大幅纠错
M4	0.3238	0.3272 (step 6)	0.3863 (step 12)	+0.0591	12–step 快线很早进入低 loss 区间，但末段略有回弹，说明已能稳定归档，尚不应把它宣称新的最佳训练配方

表 24: 四组已归档训练工件在关键 checkpoint 上的变化。

这个比较给出一个相当实际的判断：当前最适合承担快线迭代的不是 Qwen 小模型，而是已经通过 continuation 进入低 loss 区间的 OmniCoder 9B。它未必天然更泛化，但它在相似乃至更短训练预算下更早进入“可微调细化”而不是“大幅纠错”的阶段，这对于昂贵的远端 NPU 资源尤其重要。M4 的轻微回弹也提醒我们：8 NPU 快线已经足以高频出工件，然而否要把 12–step 配方继续拉长，仍需后续 benchmark 回评决定。

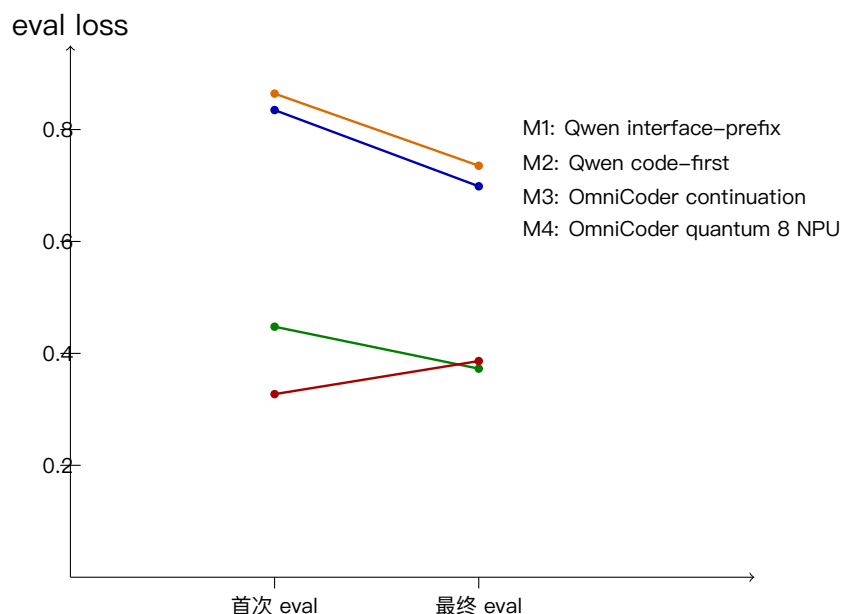


图 9: 四组归档训练工件在首次 eval 点与最终 eval 点的变化。

4.4 8-NPU 快迭代工件

`.huanxin_jobs/omnicoder-8npu-fast-sft-20260409T065250Z.json` 与回收到本地的 `outputs/omnicoder9b-quantum-generalization-sft-8npu-fastiter-20260409T1451CST/metrics.json` 是当前最重要的远端快迭代工件组合。它们至少证明了以下事情：

运行通过独立作业元数据被真实发起，而不是口头计划；

远端命令包含 `torchrun--nproc_per_node=8`，说明是 8-NPU 启动而非单卡假装分布式；

运行前会先从 S3 同步最新 `training` 目录中的关键文件，说明“本地改完、远端还是旧脚本”的问题已经被显式纳入控制面；

运行数据源与 adapter 起点都明确可追踪：训练/验证使用严格量子 holdout，adapter 起点来自已验证的 OmniCoder 2-NPU continuation 结果。

本地 `metrics.json` 已确认该 run 真正完成了 12 step，而不是只启动未收尾；

`final_eval.loss=0.3863`、`final_eval.perplexity=1.4715` 已经可以作为训练级归档指标引用。

但这个工件尚不足以支撑以下表述：

尚不足以表述为“当前 8 NPU 配方已经在严格 holdout benchmark 上取得新的 pass@1 最佳”；

尚不足以表述为“当前 8 NPU 训练已经优于 OmniCoder 2 NPU continuation 的全部下游效果”；

尚不足以表述为“GRPO 已在 8 NPU 上获得稳定收益”，因为本次 run 仍是 SFT continuation。

因此，本报告把它定位为两层证据：一层是“远端启动与同步链路已完整打通”，另一层是“8 NPU 严格量子 holdout SFT 已能形成最终训练指标”。下一步是补齐该 adapter 对应的独立下游 benchmark 回评工件。

字段	当前值	含义
job id	omnicoder - 8npu - fast-sft-20260409T065250Z	当前远端快迭代作业的唯一标识
PID	287095	远端父进程已经真实分配
output dir	outputs / omnicoder9b-quantum-generalization-sft - 8npu - fastiter-20260409T1451CST	已从 S3 回收到本地的归档目录
world size	8	由 8 进程 torchrun 启动决定
训练/验证样本	1024 / 504	严格量子 holdout 规模已进入真实 8 NPU run
completed steps	12	本轮 timeboxed 快线已完整执行到设定上限
最终评测	loss=0.3863, ppl=1.4715	训练级最终指标已形成 A 级工件
可训练参数	29,097,984	当前 LoRA continuation 配方实际打开的训练参数量
数据源	严格量子 holdout	使用严格量子 holdout 的 train/eval 切分
初始化	既有 continuation adapter	起点来自已归档的 2-NPU OmniCoder 结果
研究方法	两个插件	verifier-guided repair 与 AST interface grounding 同时启用

表 25: 当前 8-NPU 快迭代归档工件能支撑的事实。

4.5 Gemma 主线当前进展与下一步

Gemma 4 26B-A4B 当前的工作重点已经相当精确。从本地工件看，真实情况是：

`artifacts/model-source-audit-gemma4-26b-a4b-it.json` 证明模型来源真实、公开且架构为 `Gemma4ForConditionalGeneration`；

`artifacts/gemma4-26b-a4b-it-local-snapshot-handoff.json` 证明本地快照完整，包含两个权重分片、processor config 与 chat template；

`training/requirements-gemma4-runtime.txt` 与 `artifacts/gemma4-26b-a4b-it-smoke-py311-stable-runtime-2026-04-10.json` 明确记录：稳定版 `transformers==4.57.1` 在 Python 3.9 与 Python 3.11 两条本地路径上都仍不能识别 `gemma4`；继续走源码安装路径则要求 `Python ≥ 3.10`；

`scripts/bootstrap_local_python311_from_cache.sh` 与 `reports/autonomous_rd_cycle_gemma4-26b-a4b-it_2026-04-10_state.json` 已证明“缺少 `Python ≥ 3.10`”这个旧瓶颈已解除：控制面 `fresh probe` 当前选中的是 `.local-python/cpython-3.11.15/bin/python3.11`，版本 `3.11.15`；

在这个 `workspace-local Python 3.11` 路径上，`training/requirements-gemma4-runtime.txt` 的 `fresh bootstrap` 尚未通过：控制面 `2026-04-10` 记录的当前阶段已从 `gemma_local_python_gate` 前移为 `gemma_runtime_bootstrap`；

同一天的本地隔离环境试验进一步明确，`runtime bootstrap` 的具体待解决点在 `git+https://github.com/huggingface/transformers.git`：受限环境下报 `Couldnotresolvehost:github.com`，放宽后的重试已能连到 GitHub；

与此同时，`training/requirements-huanxin-cpu.txt` 的稳定 wheel 栈已经在同一条 `Python 3.11` 路径上安装成功，但对应 `smoke` 停在 `runtime_compat ("Transformersdoesnotrecognizethisarchitecture")`，这说明下一步工作是获得比 `4.57.1` 更新的 `Gemma 4-capable Transformers` 源码/runtime；

当前 trainer 仍主要是 text-only causal LM 路径，而 Gemma 4 是 conditional generation 路径，因此 runtime 问题解决之后，还有 backend 适配问题。

检查点	状态	说明
模型来源审计	通过	model_id=google/gemma-4-26B-A4B-it, config_model_type=gemma4
本地快照校验	通过	权重分片、index、processor、chat template 全部存在
Python 3.9 + 稳 定栈	待升级	transformers==4.57.1 尚不能识别 gemma4, 需升级版本
本地解释器 gate	通过	fresh probe 当前选中 .local-python/cpython-3.11.15/bin/python3.11=3.11.15
runtime boot- strap	进行中	py311 环境下正在获取 huggingface/transformers 最新源码
Python 3.11 + 稳 定栈	待升级	wheel 安装已成功, smoke 需要更新版本的 Transformers 以识别 gemma4
Homebrew 补装 路径	历史受限	旧的 brewinstallpython@3.11 路径曾因网络限制未成功，当前已有 替代方案
conditional- generation backend	待完成	当前训练器还需更完整的 processor-aware 路径

表 26: Gemma 4 26B-A4B 主线各环节当前进展。

这说明当前 Gemma 主线已经不再停留在“缺少 Python ≥ 3.10 ”的阶段，也不再停留在“普通 wheel 栈能否安装”这类泛化问题上，而是推进到了更靠后的 Gemma 4 专用 runtime 源码/ bootstrap 与 backend 适配。项目管理上的积极信号是：问题持续收敛，下一步已经非常具体。当前最小可行动作是先获得一份比 4.57.1 更新、且真正识别 gemma4 的 Transformers 源码/ runtime，然后将进展从 gemma_runtime_bootstrap 推进到 gemma_smoke / conditional-generation backend。

4.6 Timeboxed Pipeline

scripts/queue_ai2_timeboxed_pipeline.sh 的价值不在于“能启动一个脚本”，而在于它把远端实验的几个最易出错点固化了：

- target 与 profile 被显式区分，不再依赖隐式默认值；
- SFT 与 GRPO 的命令可以从同一控制面生成；
- 快 profile 默认 2 NPU，标准 profile 默认 8 NPU；
- target 不同，默认模型名、数据集、adapter 初始化与输出目录也会跟着变化。

从报告角度，这表明远端实验的每一次调试都可以落回到某个明确脚本、某个明确 profile、某个明确目标模型的配置问题，而不是不可复述的口头事件。

4.7 小结

当前算力与环境层可以得出三个结论：

- 历史稳定训练工件已经足以证明分布式训练路径是可用的；
- 当前 8-NPU 快迭代已经完成真实训练并归档工件，不再是纸面计划；
- Gemma 主线的 runtime 适配目标明确，问题已收敛到本地 runtime/backend 的具体依赖。

5 研发架构（软件架构）

5.1 Local-first 控制面

当前研发架构的一个核心原则是：先在本地验证，再通过 S3 relay 与 Huanxin shell 把已验证代码送到 ai2。这比“直接在浏览器里不断手改远端文件”更慢一些，但它换来了极其关键的可复用性与可追溯性。

这一控制面可以概括为如下五层：

本地代码与测试层：training/、evals/、tests/；

数据与工件层：data/generated/、outputs/、reports/、artifacts/；

远端同步层：scripts/push_to_s3.sh、scripts/ai2_sync_from_s3.sh；

远端运行层：scripts/ai2_shell.sh、scripts/ai2_job.sh、scripts/queue_ai2_timeboxed_pipeline.sh；

研究与汇报层：research/papers/ 与 research/RESEARCH-REPORT.tex。

5.2 ai2 远端工作目录与资产分布

为了让组会与交接时不出现“文件在哪里”的疑问，本节把 ai2 上的资产分布按基座模型、数据集、微调产物、代码入口四类列清。

5.2.1 基座模型

路径（相对于项目根目录）	模型主线	说明
models/Qwen2.5-1.5B-Instruct	稳定基线	最小可运行的 Qwen 模型，用于基线测试与 preflight 校验
models/OmniCoder-9B	快迭代主线	OmniCoder 9B continuation 的底座，当前快线所有 adapter 都基于此
models/gemma-4-26B-A4B-it	MoE 研究主线	Gemma 4 26B-A4B MoE 模型，用于 router warmup 与 ESFT 实验（runtime 适配中）

表 27: ai2 上的基座模型。绝对路径为 /root/root/work/quantum-gpt/models/<model-name>。

5.2.2 数据集

5.2.3 微调产物（adapter）

5.2.4 代码入口与控制面脚本

数据集类别	路径 (相对于 data/generated/)	规模	说明
严格量子 holdout	omnicoder-quantum-generalization-holdout-v1/ (train.jsonl, eval.jsonl, manifest.json)	1024/504	当前 OmniCoder 8-NPU fastiter 的主训练/评测集
混合域 holdout	omnicoder-generalization-holdout-v1/ (train.jsonl, eval.jsonl, manifest.json)	1440/504	量子+软件双域泛化评测
混合宽口径 SFT	omnicoder-template-large-v1/*	2208/552	SFT 主干训练数据池
量子宽口径 SFT	omnicoder-quantum-large-v1/*	1536/504	SFT 量子专项数据池
Gemma 路由 warmup	quantum-paper-router-warmup-v1/ messages/ (.._train.jsonl, .._valid.jsonl)	42/5	论文级路由预热数据

表 28: ai2 上的数据集。绝对路径前缀为 /root/root/work/quantum-gpt/data/generated/。

产物类别	路径 (相对于 outputs/)	当前状态	说明
Qwen 8-NPU interface-prefix	interface-prefix-semantic-v4 - 8npu - true20 - e2 - 20260326T1627CST/	已归档	Qwen 基线 8-NPU 训练产物
Qwen 8-NPU code-first	codefirst - semantic - mix75 - 8npu - true20 - e2 - 20260326T2118CST/	已归档	Qwen prompt 变体对比实验
OmniCoder 2-NPU continuation	interface - prefix - omnicoder9b - semantic - v4 - 2npu - true20 - 20260329T2219CST/	已归档	当前快线 continuation 的起点 adapter (adapter/ 子目录)
OmniCoder 8-NPU strict quantum	omnicoder9b - quantum - generalization - sft - 8npu - fastiter-20260409T1451CST/	已归档	当前最佳 adapter , strict quantum override 2/4 (adapter/ 子目录)

表 29: ai2 上的微调产物。绝对路径前缀为 /root/root/work/quantum-gpt/outputs/。每个目录下的 adapter/ 子目录包含 LoRA 权重, metrics.json 包含训练指标。

类别	关键相对路径	当前作用	说明
训练代码入口	training/qwen_sft_peft.py training/grpo_trainer.py training/research_plugins.py training/inspect_moe_target_modules.py	SFT / RL / plugin / MoE 方法入口	组会讨论“算法怎么做”时，应直接对应到这些实现路径
评测代码入口	scripts/run_hf_pass1_eval.py scripts / summarize_override_scorecard.py evals/runner/run_eval.py	生成 scorecard 与 override summary	所有 headline 级 benchmark 数字都应能追溯到这里生成的工件
远端控制面入口	scripts/ai2_job.sh scripts/queue_ai2_timeboxed_pipeline.sh scripts/ai2_sync_from_s3.sh scripts/ai2_push_results_to_s3.sh	ai2 作业启动、同步与结果回收	组会里谈到“焕新上怎么跑”时，应落到这些脚本

图 10 以目录树形式汇总上述四类资产在 ai2 上的位置关系。

```

/root/root/work/quantum-gpt/ (ai2 唯一工作目录)
  models/ ← 基座模型
    Qwen2.5-1.5B-Instruct/ ← 稳定基线
    OmniCoder-9B/ ← 快速迭代底座
    gemma-4-26B-A4B-it/ ← MoE 研究主线
  data/generated/ ← 数据集
    omniscoder-quantum-generalization-holdout-v1/ ← 严格量子
    omniscoder-generalization-holdout-v1/ ← 混合域
    omniscoder-template-large-v1/ ← 宽口径 SFT
    quantum-paper-router-warmup-v1/ ← Gemma 路由
  outputs/ ← 微调产物 (每个子目录含 adapter/ 与 metrics.json)
    ...omniscoder9b-...-2npu-.../adapter/ ← continuation 起点
    ...omniscoder9b-...-8npu-fastiter-.../adapter/ ← 最佳 (2/4)
  training/ ← SFT / GRPO / plugin 训练入口
  scripts/ ← ai2 作业启动、S3 同步、评测
  evals/ ← 评测 runner 与 benchmark 定义

```

图 10: ai2 远端工作目录树。所有操作从此根目录执行。

5.3 控制面组件与职责边界

对于实际研发而言，当前控制面已经能够把问题精确定位到具体组件。表 31 给出当前主要组件及其职责边界。

这个分层的直接收益是问题定位更加精确。例如，本轮 ai2 状态可以明确区分：Safari keepalive 正常工作、8-NPU fast SFT 的 durable job 元数据已存在、本机浏览器 daemon 需要重启。这种精确的分层使得运维效率大幅提升。

5.4 自治研发循环

scripts/run_autonomous_rd_cycle.py 代表的是控制面的“叙事层”。它将一个研发循环明确拆成多个 stage，例如本地评测 gate、holdout integrity、模型来源审计、快照校验、paper router warmup、remote launcher gate 等，并把状态输出成 reports/autonomous_rd_cycle*.json 与命令单工件。

组件层	代表脚本或工件	正常产物	定位作用
Safari keepalive	scripts / huanxin_safari_keepalive.sh	周期性 train-dev 保活日志	确认 Safari 会话保持活跃
浏览器 daemon	browser_automation_huanxin_browser_daemon.js	- 本地健康端 / 口 + shell 通道	确认本机 Playwright 通道就绪
远端 durable job	scripts/ai2_job.sh + .huanxin_jobs/*.json	job id、PID、日志路径	确认作业已真实发起并可追踪
时间盒 pipeline	scripts / queue_ai2_timeboxed_pipeline.sh	统一 profile 与命令生成	统一配置管理，每次运行可追溯到具体参数
S3 relay	scripts / push_to_s3.sh、scripts / ai2_sync_from_s3.sh	关键训练脚本对齐	确保远端运行的脚本版本与本地一致
报告与证据索引	research / RESEARCH - REPORT.tex、E1	单一汇报口径	保证项目汇报的一致性与可追溯性

表 31: 当前研发控制面的主要组件与故障定位边界。

这类脚本的价值在于，它把”接下来该做什么”也变成了工件的一部分。即使团队交接或上下文切换，当前主线仍然可以从 state 文件中恢复。

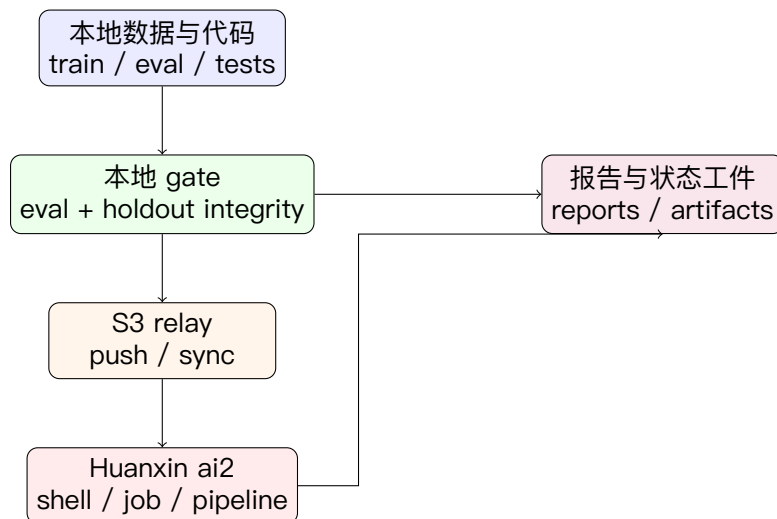


图 11: 当前 local-first 研发闭环的控制面结构。

5.5 训练器与插件的分层设计

如果把每个研究想法都直接写进 `qwen_sft_peft.py` 或 `grp0_trainer.py`，训练器会很快变得难以维护。当前仓库通过 `training/research_plugins.py` 维持分层，原因有三：

主干 trainer 应该稳定，方法实验应该可插拔；

报告中必须能说清“哪些是已稳定主线，哪些只是研究插件”；

远端快迭代时，最常同步的只是几个关键训练文件与 plugin，不必整仓库胡乱覆盖。

5.6 S3 Relay 同步机制

远端连接与操作的架构原则是：浏览器只负责进入环境与 shell 终端；代码传输与文件更新统一走 S3 relay。

这个设计避免了两个典型问题：

浏览器页面状态波动时，代码同步也跟着不稳定；

远端脚本版本容易和本地脱节，导致“本地修好了，远端还是旧 trainer”。

当前 8-NPU 快迭代作业元数据恰好证明了这一点：其命令开头先执行 `rclonecopy` 从 S3 覆盖同步若干关键训练脚本，然后才进入 `torchrun`。这正是“控制面把同步与训练显式串起来”的体现。

5.7 唯一正式报告机制

研究报告不是最终才补写的文档，而是研发架构的一部分。原因很简单：

如果没有唯一正式报告，组会口径就会漂移；

如果没有单一 PDF 产物，警告清理、版式统一、证据索引都无从谈起；

如果没有“唯一报告 + 唯一证据索引”，下一轮迭代很容易再次出现造口径、混口径、漏工件的问题。

因此，`research/RESEARCH-REPORT.tex` 的地位不低于 `training/grpo_trainer.py` 或 `scripts/queue_ai2_timeboxed_pipeline.sh`。它并不是项目的附庸，而是项目能否进行高质量汇报与决策的必要基础设施。

5.8 小结

当前软件架构的成熟点不在于“花哨”，而在于三个词：分层、留痕、可恢复。分层体现在 trainer 与 plugin 分离；留痕体现在 outputs、reports、artifacts、job meta 齐全；可恢复体现在 autonomous cycle state 与唯一正式报告并存。

6 测试结果

核心结论：参考解基准 26/26 全部通过；OmniCoder 8-NPU adapter 在协议一致的严格量子 holdout 上把 override 从 0/4 推进到 2/4——项目首个经过严格验证的真实 benchmark 收益。训练主干稳定，Gemma 主线推进到 `gemma_runtime_bootstrap` 阶段。

6.1 核心结果

表 32 汇总了当前最重要的 headline 数字。Qwen2.5-1.5B clean baseline 在 4 个严格量子未见任务上全部未通过；经过 8-NPU 分布式训练的 OmniCoder adapter 在完全相同的评测协议下通过了其中 2 个。两组实验共享相同的 4 个 override task、`prompt_version=v2`、`prompt_style=repair_focused`、`include_reference_candidate=false` 与 `token_budget_preset=quantum_heavy`，因此 0/4 → 2/4 是协议一致的真实收益，不是测试漂移。

模型 / 口径	override 通过	override 总数	通过率	说明
Qwen2.5-1.5B baseline	0	4	0%	严格量子未见任务的基础地板
OmniCoder 9B base	—	—	—	模型快照已恢复（18G），待 ai2 runtime 升级后补齐
OmniCoder 9B 8-NPU adapter	2	4	50%	已有独立 scorecard，8-NPU 快线产生了真实收益

表 32: 严格量子 holdout 上的 override-only 对比（headline 级）。

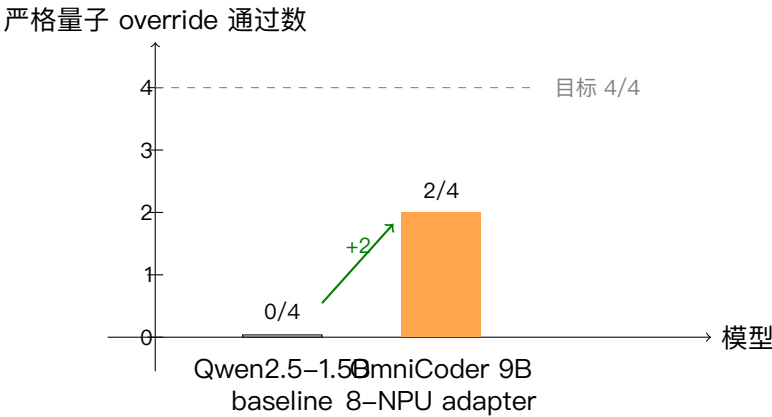


图 12: 严格量子 holdout 上的 override-only 成绩对比。OmniCoder base 9B 的第三根柱子待 ai2 runtime 升级后补齐。

表 33 进一步展示逐题变化——提升落在相位估计算法（接口命中）和超密编码协议（语义闭环）两个维度上，两个尚未通过的任务也各自有可量化的推进。

6.2 评测方法与可信度

上述结果由一条固定的自动化评测流水线产生，分四步执行：

1. **生成 prompt**——`evals/runner/prepare_prompts.py` 为每个 task 生成 run directory，写出 `manifest.json`、`candidate-map.json`、system prompt 与逐题 prompt。

任务	Qwen baseline	OmniCoder adapter	变化说明
quantum_gate_alias_normalization	未通过	未通过	API 正规化与非法输入约束待完善
quantum_phase_estimation_circuit	未通过	通过	从”缺失函数”推进到完整通过
quantum_qaoa_maxcut	未通过	未通过	从”缺接口”推进到”局部逻辑待修正”
quantum_superdense_coding	未通过	通过	协议 round-trip 已闭环

表 33: 四个严格量子 override 任务的逐题对比。

- 模型推理**——scripts/run_hf_pass1_eval.py 让 base model 或 adapter 对每个 task 各生成 1 个候选代码 (pass@1)。
- 执行测试**——evals/runner/run_eval.py 逐题加载测试模块，把候选代码放进真实执行环境里运行 task 自带的 tests。
- 汇总 headline**——scripts/summarize_override_scorecard.py 从 scorecard.json 中只抽取 source=override 的条目，得到 strict unseen benchmark 的 override-only headline。

整个流水线的评判标准是自动化的代码执行结果——不是字符串比较，也不是人工主观判断。每一轮评测的 run protocol (prompt 版本、候选策略、预算配置) 都固化在 manifest.json 中，任何人都可以用同一份 manifest 独立复现同一组结果。

scorecard.json 同时包含 reference (参考解验证) 与 override (模型在严格未见任务上的真实求解) 两类条目。本报告的 headline 只取 override 子集；参考解条目仅用于确认评测链路健康 (见下节)。因此，像”24/26”这样的整表数字只能作为调试辅助，不进入正式结论。

为了确保对比公平，Qwen baseline 与 OmniCoder adapter 使用完全相同的 4 个 override task 与 run protocol。OmniCoder base 9B 的同协议回评中，模型快照已恢复到 18G，当前待 ai2 runtime 升级 (transformers==4.44.0 需升级到可识别 qwen3_5 的版本)，补齐后将形成完整的 base-vs-adapter 三方对比。

6.3 参考解健康度

参考解 26/26 全部通过是后续一切模型评测的前提——如果参考解本身通不过，后续任何模型结果都不可信。本轮执行 python3evals/runner/run_eval.py (完整输出固化为 artifacts/reference_eval_2026-04-09.txt)，结果如下：

分组	通过数	总数	说明
总计	26	26	评测器、参考解与任务目录整体健康
量子任务	13	13	包括 gate alias、QPE、QAOA、superdense coding 等关键任务
软件任务	13	13	包括 workspace repair / smoke 两个 workspace 任务

表 34: 参考解健康度结果。

26/26 全部通过意味着后续任何模型未通过的任务都可以归因为模型能力本身，而不是评测器缺陷。

6.4 详细结果分析

6.4.1 OmniCoder 8-NPU 评测工件

最新 8-NPU OmniCoder adapter 已有训练级 `metrics.json` 与独立 benchmark `scorecard` 两份配套工件（分别位于 `outputs/omnicoder9b-quantum-generalization-sft-8npu-fastiter-20260409T1451CST/` 与 `evals/runs/omnicoder-quantum-generalization-holdout-8npu-fastiter-eval-20260409T122347Z/`）。`override-only` 结果为通过 2 / 总数 4，通过率 50%。

整份 `scorecard` 还给出 24/26 的全量数字（reference 22/22 + override 2/4）。表 35 列出了三种统计口径的区别。

统计口径	通过数	总数	解释
整份 <code>scorecard</code>	24	26	包含 22 个 reference 条目，只适合做调试记录
reference 子集	22	22	说明评测器链路正常
override 子集	2	4	唯一可代表 strict unseen benchmark 改善的 headline

表 35: 同一份 `scorecard` 的三种统计口径。

6.4.2 Qwen 严格量子 holdout 基线

Qwen2.5-1.5B 在严格量子 holdout 上的 4 个 `override` 任务均未通过，这为 OmniCoder adapter 的 2/4 提供了对比起点。

待提升任务	待提升方面	说明
<code>quantum_gate_alias_normalization</code>	大小写规范	已能识别映射方向，需进一步完善 canonical casing
<code>quantum_phase_estimation_circuit</code>	接口补全	需补全 <code>phase_estimation</code> 函数接口
<code>quantum_qaoa_maxcut</code>	接口补全	需补全 <code>maxcut_cost</code> 函数接口
<code>quantum_superdense_coding</code>	协议闭环	编解码映射的 round-trip 一致性待完善

表 36: Qwen 基线在严格量子 holdout 上的逐任务分析。

待提升 category	数量	对方法设计的含义
<code>algorithm_implementation</code>	2	核心量子算法缺失 required function，首要提升方向是把算法落成可调用接口
<code>api_normalization</code>	1	gate alias 大小写正规化未通过，需接口锚定与 canonical formatting 显式训练
<code>reasoning</code>	1	superdense coding round-trip 未闭合，需协议级语义闭环训练

表 37: Qwen 严格量子 holdout override 的 category 级待提升分布。

这些分析表明，baseline 阶段的主要提升方向是**接口命中与结构性对齐**（API 命名不稳、算法函数缺失、协议逻辑未闭环），而不是单纯的推理链条不够长。后续算法工作中的 AST-anchor、repair curriculum、clause-aware reward 直接对应这些方向。

6.4.3 接口层细节

任务	必需接口 / 关键语义	差距类型	分析细节
quantum gate _ alias normalization	- 需要实现 <code>normalize_gate_ sequence(gates)</code> ; 要把 <code>h/ hadamard/cx/CNOT/Pauli_X</code> 等别名正规化到 <code>H/CX/X</code> , 并对 非法输入抛出 <code>ValueError</code>	断言未通过	期望 <code>['H','CX','X']</code> , 实际得到 <code>['H','cx','x']</code> , 映射方向正确 但 <code>canonical casing</code> 缺失
quantum _ phase estimation circuit	- 需要实现 <code>phase_estimation _ phase _ from _ measurement</code> 。测试覆盖 <code>round-trip</code> 、边界 <code>phase</code> 与离散 化取整	接口缺失	直接缺失 <code>phase_estimation</code> , 差距在接口存在性而非数值精度
quantum _ qaoa maxcut	- 需要实现 <code>maxcut _ cost, brute _ force _ maxcut, qaoa _ cost _ landscape</code> ; 测试覆盖 <code>tri- angle/ line graph</code> 与排序规 则	接口缺失	直接缺失 <code>maxcut_cost</code> , 尚未命 中最基础的 <code>required interface</code>
quantum superdense_coding	- 需要实现 <code>encode _ message(bits)</code> 与 <code>decode _ message(operation)</code> ; 四个映射必须 <code>round- trip: 00->I, 01->X, 10-> Z, 11->XZ</code>	语义映射偏差	<code>encode messagefailedfor00</code> , 协议 语义映射需进一步对齐

6.4.4 评测覆盖度

严格量子 holdout 当前有 4 个 override task, 覆盖了整个 eval split: 每个 task 对应 126 条 eval 样本, eval prompt family 共 4 个且与训练侧完全不重叠, category 分布为 2 个 `algorithm_implementation`、1 个 `api_normalization`、1 个 `reasoning`。因此存在 `small-N` 的限制, 但不存在“只挑了偶然样本”的问题——4 个 task 各代表一种关键能力维度。

测试信号	当前证据	研发价值	后续观察指标
API 规范性	<code>gate alias</code> 大小写 待提升	检验接口锚定方法 的格式偏差消减效 果	正规化任务是否变为可部分通过
函数存在性	两个算法任务缺 失关键函数	检验 <code>trainer</code> 能否先 学出函数壳与接口	“缺失函数”是否推进到“逻辑待修正”
协议闭环性	<code>superdense cod- ing round-trip</code>	区分写量子门与理 解协议逻辑	<code>encode/decode</code> 能否同时通过
参考地板	26/26 全通过	证明评测器本身无 缺陷	后续回退均可归因到模型/数据 缺陷

表 39: 当前测试信号的研发价值。

6.5 训练指标

除了 strict holdout 上的 pass/fail 结果, 四组训练运行的 step-wise 指标进一步印证了训练主干的稳定性。

OmniCoder 2 NPU run 的训练初值已远低于 Qwen 参考 run, 且 step 10 与 step 20 的 eval 点持续改善。最新 8-NPU strict-quantum fastiter 的最佳 eval 点出现在 step 6 (`pp1=1.3870`), step 12 有所回落 (`pp1=1.4715`), 说明训练闭环可运行并产出收益, 但 12-step 配方尚需进一步调优。核心结论: **OmniCoder continuation 是当前最有效率的快速迭代底座**, 训练 loss 和 strict holdout 评测两方面均有实证支持。

运行	step 1 train loss	中间 eval 点	最终 eval 点	支持的结论
Qwen 8 NPU true20-e2	3.2799	step 10: eval _ loss = 0.8348, ppl=2.3044	step 20: eval _ loss = 0.6986, ppl=2.0110	8 NPU SFT 主干在 20 step 内稳定下降
Qwen 8 NPU code-first mix75	3.2799	step 10: eval _ loss = 0.8642, ppl=2.3731	step 20: eval _ loss = 0.7353, ppl=2.0862	prompt/data 变体可形成稳定工件
OmniCoder 9B 2 NPU true20	1.0787	step 10: eval _ loss = 0.4477, ppl=1.5647	step 20: eval _ loss = 0.3727, ppl=1.4517	continuation 路线持续优于 Qwen 参考工件
OmniCoder 9B 8 NPU strict-quantum fastiter	0.3238	step 6: eval _ loss = 0.3272, ppl=1.3870	step 12: eval _ loss = 0.3863, ppl=1.4715	严格量子 holdout 已进入真实 8 NPU 训练闭环

表 40: 四组稳定训练工件的 step-wise 摘要。

6.6 口径说明

本节汇总三类易混淆数字的口径定义，供核实与引用。

6.6.1 24/26 整表数字

仓库中最新的 24/26 scorecard 混合了参考解候选与模型候选（reference 22/22 + override 2/4），不应直接充当 headline。本报告优先使用：参考解健康度 26/26、Qwen override 0/4、OmniCoder adapter override 2/4、稳定训练工件的 loss/perplexity。

6.6.2 Delivery Gate 口径

evals/runs/omnicoder-delivery-pass1-v1/scorecard.json 包含 25 个条目（15 通过 / 10 未通过），但 delivery gate 的真实任务集由 manifest.json 定义，仅 10 个。

统计口径	条目数	可做 headline	解释
整份 scorecard.json	25	否	混合了 reference 与 gate 任务
manifest.json gate 任务	10	是	真正的 delivery gate 目标
按 manifest 过滤后	0/10	是	该 run 在全部 gate 任务上均未通过

表 41: delivery gate 中 manifest 与 scorecard 统计口径的区别。

维度	manifest gate 分布	过滤后结果	说明
任务总数	10	10	由 manifest.json 明确定义
domain 分布	量子 5, 软件 5	量子 0/5, 软件 0/5	两个 domain 均未通过 gate
category 分布	10 个 category 各 1 个	覆盖全部 gate	适合作为基础底座能力起点的基准记录

表 42: delivery gate 的真实任务分布与过滤后结果。

6.6.3 8-NPU 快迭代工件完整度

当前 8-NPU 快迭代已具备训练工件 (metrics.json) 与配套 benchmark scorecard 两份成对工件，真实 8-NPU 启动已发生并完成 12 step，strict override 2/4 有独立 scorecard 支撑。2/4 是一个有明确归档价值的中间结果，后续迭代以此为基线继续推进。

6.7 Gemma 测试状态

Gemma 4 26B-A4B 已完成模型来源审计、本地快照校验、消息级 router warmup 数据准备，以及 workspace-local Python 3.11 gate。控制面当前推进到 gemma_runtime_bootstrap——在 Python 3.11.15 + transformers==4.57.1 栈上需升级到 Gemma 4-capable 版本的 Transformers，随后进入 conditional-generation backend 对接。数据与控制面已就绪，当前工作集中于这一具体的软件依赖环节。

6.8 小结

- **核心结论**——OmniCoder 8-NPU adapter 在协议一致的 strict quantum holdout 上把 override 从 0/4 推进到 2/4，项目首个经过严格验证的 benchmark 收益已确认。
- **评测可信度**——评测器直接执行真实 tests，不依赖字符串比较或人工判断；参考解 26/26 证明评测地板可靠；所有对比共享同一 run protocol。
- **待补齐**——OmniCoder base 9B 同协议回评中，模型快照已恢复，待 ai2 runtime 升级；剩余 2 个未通过任务已有具体的接口与逻辑提升方向。
- **训练主干**——四组训练工件表明 OmniCoder continuation 是当前最有效率的快迭代底座。
- **Gemma 主线**——runtime 适配收敛到具体软件依赖环节，下一步方向明确。

7 总结与下一步

7.1 当前进展总览

图 13 按三个阶段汇总本项目截至 2026-04-10 的进展状态。

已完成	数据纪律体系、严格 holdout 校验、参考解 26/26 SFT/GRPO/plugin 算法栈、四组训练工件归档
推进中	OmniCoder strict override 2/4, 正在向 4/4 推进 Gemma 4 runtime 适配 (Python 3.11 已就位)
规划中	Gemma MoE router warmup → ESFT → 正式 demo PDF 级路由数据补齐、GRPO 8-NPU 正式 run

图 13: 当前项目进展总览：已完成、推进中、规划中三个层级。

7.2 核心结论

截至 2026-04-10，本项目已取得以下五方面实质进展：

量子编码大模型研发闭环已经成型——数据、训练、评测、远端控制面、研究插件和报告工件形成完整链路；

严格 holdout 数据体系已足够干净，后续任何提升都更有可能是真实能力提升，而不是评测泄漏；

最新 OmniCoder 8-NPU adapter 已完成独立回评，并在与 Qwen clean baseline 协议一致的 strict quantum holdout 上把 override 从 0/4 推进到 2/4——这是项目首个经过严格验证的 benchmark 收益；

OmniCoder continuation 是当前快迭代的高效底座，Gemma 4 是更具研究价值的 MoE 主线目标，两条线互为补充；

OmniCoder base 9B 的 strict quantum 公平基线当前仍未补齐，但缺口已精确收敛到”ai2 上 /usr/bin/python3+transformers==4.44.0 无法识别 qwen3_5”这个 runtime 环节，而不是实验协议不一致；

Gemma 主线的 runtime 适配范围已明确收敛到具体的软件依赖问题（需要 Gemma 4-capable Transformers 版本），下一步工作方向清晰。

7.3 下一步工作排序

以”最快提升真实研究产出”为标准，下一步工作排序如下：

先把 ai2 的 OmniCoder runtime 升级到 Qwen3.5-capable Transformers 栈，并补齐缺失的 strict quantum base benchmark，避免后续所有 adapter 对比继续缺少同模型公平地板；

在已就位的本地 Python 3.11 环境上完成 Gemma runtime 适配，获得可识别 gemma4 架构的 Transformers 版本，打通 Gemma smoke test；

保持 OmniCoder 快线持续迭代严格量子 holdout，推动 override 从 2/4 向 4/4 推进；

围绕 quantum_gate_alias_normalization 与 quantum_qaoa_maxcut 两个待通过任务做 targeted continuation 与 plugin 消融实验；

补齐 paper router 的 PDF 级工件，使 Gemma 路由 warmup 从消息级预热升级为完整论文级数据；

持续固化 Huanxin 保活、状态探测与远端 job 管理脚本，提升远端运维自动化水平。

7.4 前沿方法储备

下表列出的是后续规划中可引入的方法储备。它们写进报告是为了说明项目的技术方向与参考路线，不是本轮已完成的实验结果。

方法方向	主要来源	当前状态	计划中的落地方式
Gemma router warmup + frequency-guided ESFT	MoE 本仓库 training/inspect - moe _ target - modules . py 与 scripts / render _ timeboxed - scaleup - commands.py	规划中	先做 router/gate 定向 warmup；获得 routing-frequency 证据后据此选择首批 expert 子集进行精细调优。
Anthropic open-source circuit tracing / attribution graphs	Anthropic 官方公开研究页面（外部参考）	规划中	在有可运行 checkpoint 之后，把严格量子 holdout 待提升样例接入 circuit tracing，定位具体的内部回路瓶颈。
Anthropic monoseman-ticity / feature dictionary 路线	Anthropic 官方公开研究页面（外部参考）	规划中	对量子代码与软件代码样本构建 feature 词典，区分不同类型的能力特征簇。
TurboQuant-style KV cache compression	本仓库 research/papers/turboquant / paper . md 与 training / turboquant.py	实验性规划	仓库已具备相关代码与推理入口开关；后续计划在服务与评测路径上做对照实验。

表 43: 规划中的前沿方法储备；这些条目均为后续计划。

7.5 业界前沿技术对标

表 44 系统梳理了近期主要 AI 机构发布的前沿技术进展，并标注本项目对每项技术的采纳状态——已实现、规划中或参考关注。这张表有两个目的：帮助领导层理解本项目在业界技术图谱中的位置，以及为后续迭代提供明确的技术引入优先级。

表 44: 业界前沿技术进展与本项目对标。

机构	技术方向	本项目状态	说明
Google	TurboQuant KV cache 压缩	已实现	仓库已具备 training/turboquant.py 与完整量化/反量化路径，服务于长上下文推理优化。

续下页

机构	技术方向	本 项 目 状态	说明
Google	Gemma 4 MoE 专家调优	规划中	基于 google/gemma-4-26B-A4B-it 的 router warmup + ESFT 方案已设计完成，待 runtime 适配后执行。
Anthropic	开源 circuit tracing / attribution	规划中	计划接入量子 holdout 待提升样例，定位模型内部回路瓶颈。
Anthropic	Agent 训练（模拟环境 RL）	规划中	与本项目 GRPO 主干方向一致；计划利用量子任务测试器作为模拟环境，扩展 agent 级强化学习。
Anthropic	单义性特征词典	规划中	对量子代码与软件代码样本构建 feature 词典，区分不同类型的能力特征簇。
OpenAI	电路稀疏性与可解释性	参 考 关 注	可解释性方向与 circuit tracing 互补，后续评估是否可用于模型能力诊断。
Meta	Llama 4 MoE 架构	参 考 关 注	本项目已在 Gemma 4 MoE 上规划了类似的 router warmup + expert 选择策略；Llama 4 的 MoE 实践可作为交叉参考。
DeepSeek	稀 疏 注 意 力 (Sparse Attention)	规划中	对长上下文量子代码推理有直接价值；计划与 TurboQuant 结合，探索注意力稀疏化 + KV cache 压缩的联合优化。
DeepSeek	MoE 专家路由优化	参 考 关 注	DeepSeek 在超大规模 MoE 训练中的路由策略可为 Gemma expert 选择提供参考。
xAI	世界模型 (World Model)	参 考 关 注	量子物理模拟场景下的世界模型思路值得长期关注，当前优先级低于代码生成主线。
智谱 AI	Agent 智能体能力	规划中	本项目的自治 R&D 循环控制器已具备 agent 特征；计划进一步增强多步推理与工具调用能力。
MiniMax	模 型 自 进 化 (Self-Evolution)	规划中	与本项目利用历史 agent 轨迹做合成监督的方向一致；计划系统化已有模型轨迹的反馈闭环。
月之暗面	Agent 协作编排 (Swarm)	规划中	当前仓库已有单 agent 自治循环；计划扩展为多 agent 协作模式，提升复杂任务的分工效率。
阿里	Qwen 系列模型底座	已实现	Qwen2.5-Coder 系列是当前 Omni-Coder 的底座模型，SFT/GRPO 流程已长期使用。
Cohere	Rerank 4 检索重排	规划中	计划在论文语料 router warmup 流程中引入检索重排，提升数据筛选质量。

续下页

机构	技术方向	本项目状态	说明
Mistral	Devstral Agent	编程参考关注	Mistral 的轻量编程 agent 架构可作为本项目 agent 能力扩展的参考。
AI21 Labs	Jamba 混合架构 (Mamba + Transformer)	参考关注	混合架构在长序列建模上的效率优势值得关注，可为后续架构探索提供方向。

图 14 汇总了上表中各项技术的采纳状态分布。

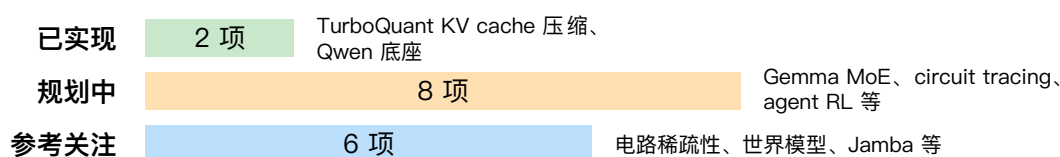


图 14: 业界前沿技术在本项目中的采纳状态分布：已实现 2 项、规划中 8 项、参考关注 6 项。

外部参考来源 以下为本节涉及的主要权威公开来源：

Anthropic open-source circuit tracing: <https://www.anthropic.com/research/open-source-circuit-tracing>

Anthropic feature dictionary / monosemanticity 路线: <https://www.anthropic.com/research/mapping-mind-language-model>

Google Gemma 官方文档: <https://ai.google.dev/gemma/docs/core>

Google Gemma 4 发布公告: <https://blog.google/technology/developers/gemma-4/>

DeepSeek 技术报告: <https://github.com/deepseek-ai>

Meta Llama 4 技术文档: <https://llama.meta.com>

阿里 Qwen 系列: <https://huggingface.co/Qwen>

AI21 Labs Jamba: <https://www.ai21.com/jamba>

7.6 展望

从工程视角看，当前仓库已经形成一个可持续演进的研发系统。下一阶段的关键是在严格 holdout 与唯一正式报告的纪律下，持续把每轮实验沉淀为可信工件，并推动 Gemma MoE 主线和 OmniCoder 快线的双轨进展。

A 附录 A：数据集与任务清单

A.1 A.1 数据集清单与用途

数据集	训练数	验证数	领域	划分策略	主要用途
omnicoder-template-large-v1	2208	552	量子+软件	prompt family disjoint	宽口径监督数据，适合作为基础 SFT 语料
omnicoder-generalization-holdout-v1	1440	504	量子+软件	task disjoint	领导汇报级混合域严格 hold-out
omnicoder-quantum-large-v1	1536	504	量子	prompt family disjoint	宽口径量子 SFT 基础集
omnicoder-quantum-generalization-holdout-v1	1024	504	量子	task disjoint	严格量子未见任务 holdout
omnicoder-quantum-focus-v1	252	48	量子偏置混合	随机拆分	快速量子偏置迭代
omnicoder-quantum-hard-v1	121	19	量子	task subset split	难例聚焦与压力测试
quantum-paper-router-warmup-v1/messages	42	5	论文消息	消息拆分	Gemma 路由 warmup 消息级输入

A.2 A.2 量子任务清单

task 目录名	任务标题	类别	备注
bell_pair_construction	Bell pair construction	circuit_construction	基础量子态构造，适合校验最小线路逻辑
circuit_depth_optimization	Greedy circuit depth optimization via ASAP scheduling	circuit_optimization	偏静态优化与调度
circuit_phase_repair	Circuit phase repair from noisy gate stream	debug_repair	偏修复类量子门序列问题
error_detection_bit_flip	Quantum Bit-Flip Error Detection	debug_repair	当前参考基准中存在，但在严格量子 holdout 清单内
gate_alias_normalization	Gate alias normalization repair	api_normalization	严格量子 holdout 验证任务之一
measurement_bug_repair	Measurement mapping bug repair	debug_repair	当前训练任务中常用的量子修复类样本源
phase_estimation_circuit	Quantum phase estimation unitary eigenvalue extraction	algorithm_implementation	严格量子 holdout 验证任务之一
qaoa_maxcut	QAOA MaxCut cost function and classical simulation	algorithm_implementation	严格量子 holdout 验证任务之一
qft_phase_pattern	QFT phase pattern	algorithm_reasoning	偏推理与公式一致性
stabilizer_tableau_update_repair	Stabilizer tableau single-qubit update repair	debug_repair	经典稳定子表修复类任务
superdense_coding	Superdense coding encode/decode	reasoning	严格量子 holdout 验证任务之一
teleportation_corrections	Teleportation correction mapping	protocol_logic	协议映射类任务

task 目录名	任务标题	类别	备注
vqe_energy_minimization	VQE energy expectation from parameterized ansatz	algorithm_implementation	训练侧重要量子算法任务

A.3 A.3 软件任务清单

task 目录名	任务标题	类别	备注
config_merge	Configuration merge semantics	data_transforms	训练集常见软件数据变换任务
docstring_contract	Docstring contract preservation	api_quality	偏接口行为与文档一致性
duplicate_logic_refactor	Duplicate logic refactor	refactor	偏重构与共享逻辑抽取
multifile_patch_conflict_repair	Multifile patch conflict repair	multifile_repair	混合 holdout 验证任务之一
off_by_one_bugfix	Off-by-one bug fix	bugfix	基础软件修复任务
parser_regression_tests	Parser regression tests	test_writing	偏测试编写能力
patch_application_conflict_resolver	Patch application conflict resolver	stateful_logic	混合 holdout 验证任务之一
retry_decorator	Configurable retry decorator with exponential backoff	design_pattern	混合 holdout 验证任务之一
session_event_log	Session event log state machine	stateful_logic	训练集中的状态机类任务
session_window_summary	Session window summary with ordered history	stateful_logic	混合 holdout 验证任务之一
tree_serialization	N-ary tree serialization and deserialization	data_structures	训练集中的结构化软件任务
workspace_patch_bundle_repair	Workspace patch bundle repair	workspace_repair	当前 26/26 参考基准中的 workspace 任务
workspace_runner_smoke	Workspace runner smoke	workspace_smoke	当前 26/26 参考基准中的 workspace 任务

A.4 A.4 严格 holdout 的 task 划分

任务	混合 holdout 角色	量子 holdout 角色	说明
quantum_bell_pair_construction	验证	训练	量子 holdout 与混合 holdout 对同一任务的定位不同
quantum_circuit_depth_optimization	训练	训练	量子训练侧稳定任务
quantum_circuit_phase_repair	训练	训练	量子修复类训练任务
quantum_gate_alias_normalization	验证	验证	当前最关键的 API 正规化测试点
quantum_measurement_bug_repair	验证	训练	修复类任务在两个 holdout 中角色不同
quantum_phase_estimation_circuit	训练	验证	算法实现型验证点
quantum_qaoa_maxcut	训练	验证	算法实现型验证点

任务	混合 holdout 角色	量子 holdout 角色	说明
quantum_qft_phase_ pattern	训练	训练	量子推理型训练任务
quantum_ stabilizer_ tableau_update_ repair	验证	训练	混合 holdout 验证点之一
quantum_ superdense_coding	训练	验证	协议推理型验证点
quantum_ teleportation_ corrections	训练	训练	协议映射型训练任务
quantum_vqe_ energy_ minimization	训练	训练	量子算法训练任务
software_config_ merge	训练	不适用	混合 holdout 训练软件任务
software_ docstring_contract	训练	不适用	混合 holdout 训练软件任务
software_ duplicate_logic_ refactor	训练	不适用	混合 holdout 训练软件任务
software_ multifile_patch_ conflict_repair	验证	不适用	混合 holdout 软件验证任务
software_off_by_ one_bugfix	训练	不适用	混合 holdout 训练软件任务
software_parser_ regression_tests	训练	不适用	混合 holdout 训练软件任务
software_patch_ application_ conflict_resolver	验证	不适用	混合 holdout 软件验证任务
software_retry_ decorator	验证	不适用	混合 holdout 软件验证任务
software_session_ event_log	训练	不适用	混合 holdout 训练软件任务
software_session_ window_summary	验证	不适用	混合 holdout 软件验证任务
software_tree_ serialization	训练	不适用	混合 holdout 训练软件任务

A.5 A.5 量子任务逐项说明

Bell pair construction 该任务要求模型生成最基础的 Bell pair 构造逻辑，看似简单，但它是量子线路构造任务中最重要的底座之一。如果连这个任务都不稳定，就很难相信模型在更复杂的多比特协议中能维持正确的门序列与状态语义。

Greedy circuit depth optimization via ASAP scheduling 这个任务体现的是“量子算法不只是写对，还要写得更优”。它要求模型理解门依赖关系、并行性与调度顺序，因此比纯 API 填空更接近工程型量子编程。

Circuit phase repair from noisy gate stream 这是典型的修复类量子任务。它要求模型从已有的错误门流中恢复出相位与归一化一致的版本，适合验证 repair curriculum 这类方法是否真的有意义。

Quantum Bit-Flip Error Detection 该任务在当前 26/26 参考基准中存在，但没有被纳入严格量子 holdout。它的存在价值是提供一个更偏纠错和 syndrome 逻辑的量子样本，为未来扩充严格量子 holdout 留出方向。

Gate alias normalization repair 这是当前严格量子 holdout 中最具代表性的 API 正规化任务。模型未通过的原因往往不是“完全不会”，而是大小写、别名映射与不合法输入处理不稳，因此它能直接检验 AST-anchor 与接口约束是否有效。

Measurement mapping bug repair 该任务专注于测量位与逻辑量子比特之间的映射。它非常适合作为训练侧的修复样本源，因为失败模式清晰、接口确定、行为检查也相对便宜。

Quantum phase estimation unitary eigenvalue extraction 这是严格量子 holdout 中最重要的算法实现任务之一。它检验模型是否能完成一个完整的经典-量子算法接口，而不是只在表面上拼接量子门名字。

QAOA MaxCut cost function and classical simulation 这个任务同样位于严格量子 holdout 的验证侧。它兼顾量子问题建模与经典代价函数计算，因此非常适合暴露“函数缺失”“接口错误”“局部逻辑不闭环”这三类系统性问题。

QFT phase pattern QFT phase pattern 更偏向算法推理和公式一致性，而不是单纯代码模板。它对模型的要求是理解结构性的相位规律，而不是记住几个函数名。

Stabilizer tableau single-qubit update repair 稳定子表更新任务位于“修复”和“符号正确性”交界处。它非常适合测试模型是否能在不重写全部逻辑的前提下，修正局部符号、轴变换与门序列影响。

Superdense coding encode/decode 这是严格量子 holdout 中唯一明确偏协议型 reasoning 的任务。它要求模型同时理解编码映射、解码映射与 round-trip 正确性，因此对“写出一个貌似合理的函数”和“真正理解协议”有很强区分力。

Teleportation correction mapping 该任务比 superdense coding 更小，但同样检验协议映射能力。它适合放在训练侧作为结构清晰、反馈明确的协议类任务。

VQE energy expectation from parameterized ansatz VQE 任务代表连续参数化量子算法方向。把它纳入训练侧，有助于避免训练集全部被修复类和离散门序列类任务主导。

A.6 A.6 软件任务逐项说明

Configuration merge semantics 这个任务检验模型处理嵌套结构、覆盖优先级与输入不变性的能力。它虽然不涉及量子概念，但对编程模型的基本软件工程素养至关重要。

Docstring contract preservation 该任务强调“行为和文档同时正确”，能测出模型是否会在重构或修复时破坏既有对外契约。这类任务直接对应真实工程中的 API 稳定性问题。

Duplicate logic refactor 重构类任务并不只是“把代码写短”。它要求模型识别共享逻辑、抽取公共函数，同时不改变既有行为，是修复类任务之外非常有价值的软件信号。

Multifile patch conflict repair 这是混合 holdout 验证侧的多文件任务。它的存在很重要，因为它迫使模型面对跨文件一致性，而不只是单文件局部正确。

Off-by-one bug fix Off-by-one 是最经典的软件 bug 类型之一。该任务虽小，却能很好衡量模型对边界条件与包含/排除语义的敏感度。

Parser regression tests 测试编写任务对代码模型极具区分力，因为它要求模型理解失败模式、整理样例并转写成自动化测试，而不是单纯实现目标函数。

Patch application conflict resolver 这类状态逻辑任务要求模型维护顺序、去重、冲突记录等多种约束，适合暴露“局部看似合理、整体状态机错误”的问题。

Configurable retry decorator with exponential backoff retry decorator 任务本身不大，但涉及参数化、异常控制与时间序列语义，是典型的软件设计模式任务。

Session event log state machine 状态机类任务对 GRPO 很有意义，因为很多候选会在局部接口正确的同时破坏状态转移。它们天然适合细粒度 reward 分解。

Session window summary with ordered history 该任务与 event log 相近，但更强调顺序性与窗口聚合。对任何希望具备“写业务逻辑”能力的代码模型，这类任务都不该缺席。

N-ary tree serialization and deserialization 树结构序列化任务能检验模型是否真正理解递归结构与逆变换，而不是只会平铺列表逻辑。

Workspace patch bundle repair 这是 workspace 模式下的重要任务，强调跨模块修复与最终 bundle 语义一致性。它接近真实代码仓修复，而不是题库式函数填空。

Workspace runner smoke 该任务作用类似最小工作区健康度检查。其重要性不在难度，而在于它让 benchmark 不再只覆盖“算法题风格”的代码生成。

A.7 A.7 核心数据集的 category 细表

数据集	category	train 计数	eval 计数	解释
omnicoder-template-large-v1	debug_repair	288	72	宽口径数据中的最大修复簇之一
omnicoder-template-large-v1	algorithm_implementation	288	72	宽口径数据中的最大算法实现簇之一
omnicoder-template-large-v1	stateful_logic	288	72	软件状态机/窗口总结类任务形成第三个大簇
omnicoder-template-large-v1	其余 14 类	各 96	各 24	保证数据面覆盖并避免长尾类别完全消失
omnicoder-quantum-large-v1	debug_repair	384	126	量子宽口径数据的最大簇之一
omnicoder-quantum-large-v1	algorithm_implementation	384	126	量子宽口径数据的最大簇之一
omnicoder-quantum-large-v1	circuit construction / circuit optimization / api_normalization / algorithm_reasoning / reasoning / protocol_logic	各 128	各 42	量子宽口径数据仍保持六类次一级能力面
omnicoder-generalization-holdout-v1	algorithm_implementation	288	0	训练侧故意保留较高比重，强调跨域算法实现
omnicoder-generalization-holdout-v1	debug_repair	96	126	验证侧更强调未见 repair 能力
omnicoder-generalization-holdout-v1	stateful_logic	96	126	验证侧也刻意施压状态逻辑泛化
omnicoder-generalization-holdout-v1	circuit construction / api_normalization / multifile_repair / design_pattern	0 或 96	各 63	验证侧由 train 中未出现或低频能力点构成迁移压力
omnicoder-quantum-generalization-holdout-v1	debug_repair	384	0	训练侧以量子修复主导
omnicoder-quantum-generalization-holdout-v1	algorithm_implementation	128	252	验证侧显著抬高算法实现压力
omnicoder-quantum-generalization-holdout-v1	api_normalization	0	126	验证侧专门放入 API 正规化能力点
omnicoder-quantum-generalization-holdout-v1	reasoning	0	126	验证侧专门放入协议/推理型能力点

A.8 A.8 核心数据集的 prompt family 细表

数据集	train family	prompt 分布	eval family	prompt 分布	解释
omnicoder- template-large-v1	六类均为	368	四类均为	138	当前最规则的 family-disjoint 大集合之一，适合作为基础 prompt 风格底座
omnicoder-quantum- large-v1	264, 252, 252, 252	264, 252,	132, 120, 120	132,	量子宽口径集在风格上近似均衡，但并非完全平均
omnicoder- generalization- holdout-v1	六类均为	240	128, 128, 120	128,	mixed holdout 的训练风格最均匀，有利于把验证压力集中在 task 与 domain，而不是训练 prompt 偏置
omnicoder-quantum- generalization- holdout-v1	176, 176, 168, 168, 168, 168	168, 168, 168	128, 124, 124	128,	strict quantum holdout 在风格上仍然保持足够均匀，因此 0/4 结果不能被解释为 prompt style 偶然失配

B 附录 B：研究方法插件逐项说明

B.1 B.1 插件接口摘要

方法	接口阶段	主要 hook	报告中应如何表述
verifier_guided_repair_curriculum	SFT + GRPO	record 增广、prompt 增广、权重调节	已实现的修复优先方法
clause_aware_verifier_reward	GRPO	reward 调整	已实现的细粒度 verifier 奖励
ast_anchor_interface_grounding	SFT + GRPO	record/ prompt/ reward	已实现的接口锚定方法
behavior_anchor_coverage_reward	GRPO	reward 调整	已实现但偏探索的方法
self_consistency_verifier_routing	SFT + GRPO	prompt/reward	已实现的一致性路由方法
uncertainty_triggered_repair_replay	SFT + GRPO	record/ prompt/ 权重/reward	已实现的不确定性 replay 方法

B.2 B.2 Verifier-Guided Repair Curriculum

该方法的主要假设是：大量代码任务的最佳学习单位不是“从零写一份全新的候选”，而是“在保持接口与主体结构的前提下，修复离通过最近的错误”。当前仓库将这一思想写入了 SFT record 与 GRPO prompt，因此其贡献不只是一个概念，而是具体可执行的训练信号调整。

B.3 B.3 Clause-Aware Verifier Reward

该方法的主要价值在于把测试器输出从稀疏的 0/1 重新切分成多个可学习条款。例如，若函数名正确但返回值格式错误，则应获得部分信用；若关键 API 缺失，则即使语法正确也不应获得过高信用。现有实现通过 verifier 反馈与行为条款的对齐，将这一思想编码到 reward breakdown 中。

B.4 B.4 AST Anchor Interface Grounding

该方法强调“先把接口做对”。对量子任务与软件任务而言，这通常意味着：

候选模块导出的函数名应与测试器期望一致；

返回值形状与类型应尽早对齐；

复杂逻辑尚未完全正确时，仍可先通过接口得分为后续优化保留学习路径。

B.5 B.5 Behavior Anchor Coverage Reward

这是典型的低成本部分信用方法。它不试图取代测试器，而是提取一组行为锚点，当候选代码至少覆盖这些锚点时给予附加分。其最大优点是计算便宜，需要注意的是 reward 容易被表面

匹配影响。因此，报告中应把它定位为探索型方法，而不是稳定主线。

B.6 B.6 Self-Consistency Verifier Routing

该方法关注的不是单个候选本身，而是多个可能解之间的内部一致性与 verifier 对齐倾向。其价值在于让模型朝着“更容易通过测试器”的内部路由偏置，而不是只学会写表面合理的文字解释。

B.7 B.7 Uncertainty-Triggered Repair Replay

当前数据规模仍然有限，尤其是严格量子 holdout 只有 12 个源量子任务中的一部分。因此，每个难任务、每次 near-miss 都很宝贵。该方法通过 difficulty、category、behavior hints、interface complexity 等信号估计样本 hardness，并对高不确定样本做 replay，从而提高训练预算利用率。

B.8 B.8 TurboQuant

TurboQuant 并不是 plugin，而是推理 runtime 方向。把它放在附录中单列，是为了防止读者把“推理时 KV cache 压缩”误读成“训练时量化微调”。当前 `training/turboquant.py` 已具备 Hadamard 旋转、按组对称量化、残差量化与 cache slice 恢复等核心组件，适合后续在服务路径与评测路径上做长上下文效率实验。

C 附录 C：工件索引与引用口径

编号	相对路径	级别	当前作用	主要包含的可引用信息
E1	artifacts/report_evidence_2026-04-09.json	A	总索引	本报告各类关键工件的集中索引
E2	artifacts/reference_eval_2026-04-09.txt	A	参考解健康度	26/26、13/13、13/13 的重新执行结果
D1	data/generated/omnicoder-template-large-v1/manifest.json	A	数据规模	2208/552、23 个宽口径任务源
D2	data/generated/omnicoder-generalization-holdout-v1/manifest.json	A	数据结构	混合 holdout 的 task/ prompt family 划分
D3	data/generated/omnicoder-quantum-large-v1/manifest.json	A	数据规模	1536/504 的量子宽口径数据
D4	data/generated/omnicoder-quantum-generalization-holdout-v1/manifest.json	A	数据结构	量子 holdout 的 8 训练 task 与 4 验证 task
D5	data/generated/omnicoder-quantum-focus-v1/manifest.json	A	快迭代子集	252/48 的量子偏置小集合
D6	data/generated/omnicoder-quantum-hard-v1/manifest.json	A	难例子集	121/19 的 hard 子集
V1	reports/omnicoder_generalization_holdout_v1_integrity.json	A	诚信校验	混合 holdout 的零重叠证明
V2	reports/omnicoder_quantum_generalization_holdout_v1_integrity.json	A	诚信校验	量子 holdout 的零重叠证明
B0	evals/runs/qwen25-quantum-generalization-holdout-clean-local/manifest.json	A	strict baseline 协议	Qwen clean baseline 的 4 task、prompt 与 token budget 定义
B1	reports/qwen25_quantum_generalization_holdout_clean_local_override_summary.json	A	基线效果	严格量子 holdout override 0/4 与详细分析
B2	reports/omnicoder9b_quantum_generalization_holdout_8npu_fastiter_eval_20260409_override_summary.json	A	最新严格量子回评	strict quantum holdout override 2/4 与详细分析
B2a	evals/runs/omnicoder-quantum-generalization-holdout-8npu-fastiter-eval-20260409T122347Z/manifest.json	A	strict adapter 协议	OmniCoder adapter 回评所用的同协议 manifest
B3	evals/runs/omnicoder-quantum-generalization-holdout-8npu-fastiter-eval-20260409T122347Z/scorecard.json	A	最新全量回评	最新 8-NPU adapter 的 24/26、reference 22/22、override 2/4
B4	reports/omnicoder9b_quantum_generalization_holdout_base_ai2_20260410_blocker.json	A	base 回评进展	2026-04-10 在 ai2 上直接确认 18G snapshot 已恢复，但 base eval 仍卡在 transformers==4.44.0 对 qwen3_5 的 runtime 不兼容
M1	outputs/interface-prefix-semantic-v4-8npu-true20-e2-20260326T1627CST/metrics.json	A	训练主干健康度	Qwen 8 NPU interface-prefix 结果

编号	相对路径	级别	当前作用	主要包含的可引用信息
M2	outputs/codefirst-semantic-mix75-8npu-true20-e2-20260326T2118CST/metrics.json	A	prompt 变体对比	Qwen 8 NPU codefirst 结果
M3	outputs/...omnicoder9b-2npu-true20-.../metrics.json	A	Omni 快线	OmniCoder 2 NPU continuation 结果
M4	outputs/...omnicoder9b-8npu-fastiter-.../metrics.json	A	Omni 严格量子快线	OmniCoder 8 NPU strict-holdout 结果
H1	.huanxin_jobs/omnicoder-8npu-fast-sft-20260409T065250Z.json	A	远端快速迭代作业元数据	8 NPU 启动命令、PID、日志路径、插件列表
G1	artifacts/model-source-audit-gemma4-26b-a4b-it.json	A-	Gemma 来源审计	model id、架构、pipeline tag、下载信息
G2	artifacts/gemma4-26b-a4b-it-local-snapshot-handoff.json	A-	Gemma 快照完整性	权重分片、processor、chat template 齐全
G3	reports/autonomous_rd_cycle_gemma4-26b-a4b-it_2026-04-09.json	A-	Gemma 当前阶段摘要	stage、进展状态、下一步命令与 warmup 计划
G4	reports/autonomous_rd_cycle_gemma4-26b-a4b-it_2026-04-09_state.json	A-	Gemma 控制面状态明细	gemma_local_python_gate、候选解释器列表、建议安装命令
G5	reports/autonomous_rd_cycle_gemma4-26b-a4b-it_2026-04-10_state.json	A-	Gemma 最新控制面状态	2026-04-10 fresh local eval/holdout 通过, 当前 stage=gemma_runtime_bootstrap
G6	artifacts/gemma4-26b-a4b-it-smoke-py311-stable-runtime-2026-04-10.json	A-	Gemma py311 稳定栈 smoke	Python 3.11 + transformers==4.57.1 处于 runtime_compat 阶段, 需升级
R1	training/requirements-gemma4-runtime.txt	A-	Gemma runtime 现实约束	Python 3.9 不足、Python $\geq$ 3.10 要求
R2	scripts/resolve_python_interpreter.py	A-	本地解释器 gate	最低版本探测、workspace-local Python probe、bootstrap 建议
A1	training/qwen_sft_peft.py	A	SFT 方法描述	adapter-init、completion-only、selective training
A2	training/grpo_trainer.py	A	RL 方法描述	四路 reward、curriculum、temperature
A3	training/research_plugins.py	A	插件架构	方法接口与加载机制
TQ1	research/papers/turboquant/paper.md	A-	方法说明	TurboQuant 的仓库内定位
TQ2	training/turboquant.py	A-	运行时实现	量化 cache 的具体实现起点

C.1 C.2 数字引用矩阵

引用数字或结论	数值/状态	是否主表可用	主要来源
混合 holdout 训练/验证规模	1440/504	是	V1, D2
混合 holdout 训练/验证 task 数	15/8	是	V1, D2
混合 holdout 训练领域分布	768 量子 / 672 软件	是	V1
混合 holdout 验证领域分布	252 量子 / 252 软件	是	V1
量子 holdout 训练/验证规模	1024/504	是	V2, D4
量子 holdout 训练/验证 task 数	8/4	是	V2, D4
量子 holdout 验证任务列表	4 个任务	是	V2, D4, B1
strict quantum 对比协议一致	是	是	B0, B2a
参考解健康度	26/26	是	E2
参考解量子/软件分布	13/13, 13/13	是	E2
Qwen 严格量子 holdout override 通过率	0/4	是	B1
OmniCoder 8-NPU strict quantum override 通过率	2/4	是	B2
OmniCoder base 9B strict quantum 同协议结果	暂缺, run-time 升级中	是, 需注明缺失原因	B4
OmniCoder 8-NPU strict quantum full scorecard	24/26 (reference 22/22 + override 2/4)	否, 需注明口径	B3
Qwen 未通过任务类别	2 算法实现 + 1 API + 1 reasoning	是	B1
OmniCoder 8-NPU strict quantum 未通过任务	gate alias + qaoa 两项未通过	是	B2, B3
ai2 OmniCoder snap-shot 状态	18G 且 model.safetensors 已存在	是, 进展说明	B4
ai2 OmniCoder base eval runtime	/usr / bin / python3 + transformers=4.44.0	是, 有限 blocker 说明	B4
Qwen 8-NPU interface-prefix 最终 loss	0.6986	是	M1
Qwen 8-NPU codefirst 最终 loss	0.7353	是	M2
OmniCoder 2-NPU continuation 最终 loss	0.3727	是	M3

引用数字或结论	数值/状态	是 否 主 表可用	主要来源
OmniCoder 2-NPU 最终 perplexity	1.4517	是	M3
当前 8-NPU 快迭代作业已启动	是	是	H1
当前 8-NPU 快迭代最终 eval loss	0.3863	是	M4
当前 8-NPU 快迭代最终 perplexity	1.4715	是	M4
当前 8-NPU 快迭代 completed steps	12	是	M4
当前 8-NPU 快迭代最佳 eval 点	step 6: 0.3272 / 1.3870	是	M4
Gemma 来源审计通过	是	是	G1
Gemma 本地快照校验通过	是	是	G2
Gemma 本地 Python gate 已通过	是	是	G5, R2
Gemma runtime bootstrap 仍未通过	是	是	G5, R1
Gemma 在 py311 + 稳定 4.57.1 上仍 runtime_compat 未通过	是	是	G6, R1
paper router message 数据规模	42/5	是	E1 (索引)、正文命令复核
paper router pdf record 行数	0/0	是	E1 (索引)、正文命令复核

C.2 C.3 训练与测试 provenance 的最小引用单元

正式研究报告最容易犯的错误，是把“一个目录名”当成证据。但在当前仓库里，真正可靠的最小引用单元其实有四类：

manifest：定义样本如何构成、有哪些任务、采用什么切分策略；

integrity report：定义某个切分是否真的满足零重叠或最小规模要求；

scorecard / override summary：定义模型在某批具体任务上的通过/失败事实；

metrics：定义训练 run 的 step-wise 与最终数值。

这张表之所以写进附录，是为了让后续任何人扩写该报告时都能遵守相同纪律：如果一个数字既不是来自 manifest，也不是来自 integrity、scorecard 或 metrics，那么它就应该先降级为背景说明，而不是直接进入主结论。

证据单元	典型文件	最适合引用的内容	不应承担的结论
manifest	data / generated / * / manifest.json; evals / runs/*/manifest.json	样本规模、task 构成、domain/ category/ prompt family 分布、切分策略、gate 任务清单	不能单独证明模型效果，也不能单独证明零重叠真实性
integrity report	reports / *integrity .json	min-eval-count、example/ task/ prompt family overlap 是否为空、manifest 声称是否兑现	不能单独证明模型效果或训练收益
scorecard / summary	scorecard .json; override summary.json	. pass/ fail、- 未通过任务列表、错误类型、分析细节	不能替代训练指标，也不能自动外推到未包含的任务
metrics	outputs/*/metrics.json	train/ eval examples、completed steps、final eval loss、step-wise trajectory	不能单独证明严格 holdout 泛化或交付能力

表 54: 本报告应优先使用的最小证据单元。

D 附录 D：关键命令与复现实验

D.1 D.1 参考解健康度复现命令

```
python3 evals/runner/run_eval.py
```

本轮完整输出已固化到：

```
artifacts/reference_eval_2026-04-09.txt
```

D.2 D.2 严格 holdout 诚信校验命令

```
python3 scripts/verify_holdout_dataset.py \  
  --train-file data/ generated/ omnocoder-quantum-generalization-holdout-v1/  
train.jsonl \  
  --eval-file data/ generated/ omnocoder-quantum-generalization-holdout-v1/  
eval.jsonl \  
  --manifest data/ generated/ omnocoder-quantum-generalization-holdout-v1/  
manifest.json \  
  --min-eval-count 500 \  
  --require-task-disjoint \  
  --require-prompt-family-disjoint
```

D.3 D.3 当前 8-NPU 快迭代作业的核心命令形态

```
torchrun --nproc_per_node=8 training/qwen_sft_peft.py \  
  --model-name models/OmniCoder-9B \  
  --adapter-init outputs/interface-prefix-omnicoder9b-\  
semantic-v4-2npu-true20-20260329T2219CST/adapter \  
  --train-file data/ generated/ omnocoder-quantum-generalization-holdout-v1/  
train.jsonl \  
  --eval-file data/ generated/ omnocoder-quantum-generalization-holdout-v1/  
eval.jsonl \  
  --output-dir outputs/omnicoder9b-quantum-generalization-\  
sft-8npu-fastiter-20260409T1451CST \  
  --device npu \  
  --max-length 512 \  
  --max-steps 12 \  
  --num-epochs 1 \  
  --per-device-batch-size 1 \  
  --gradient-accumulation-steps 2 \  
  --eval-steps 6 \  
  --log-steps 1 \  
  --train-on-completions-only \  
  --research-methods verifier_guided_repair_curriculum \  
ast_anchor_interface_grounding
```

D.4 D.4 Gemma 本地最小下一步实验

```
python3.10 -m venv .venv-gemma4-smoke-py310
.venv-gemma4-smoke-py310/bin/python -m pip install --upgrade pip
.venv-gemma4-smoke-py310/bin/python -m pip install -r training/
requirements-gemma4-runtime.txt
.venv-gemma4-smoke-py310/bin/python training/huanxin_cpu_smoke.py \
    --model-name models/gemma-4-26B-A4B-it \
    --dataset data/seed/splits-auto-seed/train.jsonl \
    --max-samples 1
```

D.5 D.5 报告编译命令

```
xelatex -interaction=nonstopmode -halt-on-error -file-line-error \
    -output-directory=research/build research/RESEARCH-REPORT.tex
```

E 附录 E：训练运行逐步指标摘录

E.1 E.1 Qwen 8 NPU interface-prefix true20-e2 逐步指标

step	epoch	train loss	eval loss / eval ppl
1	1	3.2799	—
2	1	1.6777	—
3	1	2.0838	—
4	1	1.2685	—
5	1	1.5479	—
6	1	1.3487	—
7	1	1.3557	—
8	1	0.8366	—
9	1	0.9643	—
10	1	0.8271	0.8348 / 2.3044
11	2	0.8889	—
12	2	0.5212	—
13	2	0.8665	—
14	2	0.7151	—
15	2	1.0403	—
16	2	0.4570	—
17	2	0.9879	—
18	2	0.6506	—
19	2	0.9430	—
20	2	0.8105	0.6986 / 2.0110

E.2 E.2 Qwen 8 NPU codefirst-semantic mix75 true20-e2 逐步指标

step	epoch	train loss	eval loss / eval ppl
1	1	3.2799	—
2	1	1.6812	—
3	1	1.9564	—
4	1	1.2709	—
5	1	1.5545	—
6	1	1.3465	—
7	1	1.3589	—
8	1	0.8452	—
9	1	1.0593	—
10	1	0.9332	0.8642 / 2.3731
11	2	0.9005	—
12	2	0.5277	—
13	2	0.8984	—
14	2	0.7236	—
15	2	1.0460	—
16	2	0.4595	—

step	epoch	train loss	eval loss / eval ppl
17	2	0.9908	—
18	2	0.7191	—
19	2	0.9696	—
20	2	0.8245	0.7353 / 2.0862

E.3 E.3 OmniCoder 9B 2 NPU continuation true20 逐步指标

step	epoch	train loss	eval loss / eval ppl
1	1	1.0787	—
2	1	1.1612	—
3	1	1.0481	—
4	1	1.2363	—
5	1	0.6473	—
6	1	0.3516	—
7	1	0.4337	—
8	1	0.3450	—
9	1	0.6122	—
10	1	0.2506	0.4477 / 1.5647
11	1	0.4053	—
12	1	0.7463	—
13	1	0.2923	—
14	1	0.4549	—
15	1	0.7051	—
16	1	0.4524	—
17	1	0.3668	—
18	1	0.2699	—
19	1	0.4104	—
20	1	0.1629	0.3727 / 1.4517

E.4 E.4 OmniCoder 9B 8 NPU strict-quantum fastiter 逐步指标

step	epoch	train loss	eval loss / eval ppl
1	1	0.3238	—
2	1	0.2493	—
3	1	0.2173	—
4	1	0.0026	—
5	1	0.0150	—
6	1	0.0077	0.3272 / 1.3870
7	1	0.0461	—
8	1	0.0085	—
9	1	0.0034	—
10	1	0.0053	—
11	1	0.0063	—

step	epoch	train loss	eval loss / eval ppl
12	1	0.0019	0.3863 / 1.4715

附录 E 的作用不是重复正文表格，而是为”趋势图与复核”提供可直接人工对照的 step-wise 数列。这样即使后续不访问原始 JSON，审阅人仍可以在 PDF 中看到训练轨迹的基本形状。这里也能直接看出：step 6 的 eval 点优于 step 12，因此当前 8-NPU fastiter 更像 timeboxed 快试验，而不是已经单调收敛到最优点的最终配方。